# Excel Shrink: An Open-Source Tool for Reducing File Bloat in Excel Spreadsheets

Alexander Aue-Johr
alexander.aue@thuenen.de
Thünen Institute for Rural Areas
Brunswick, Germany

Hardy Pundt
hpundt@hs-harz.de
Harz University of Applied Sciences
Wernigerode, Germany

## ABSTRACT

Excel files are widely used for data exchange across various domains. However, as spreadsheets grow in complexity, they often suffer from file bloat, leading to significant challenges in file size and performance management. We investigated Excel files from the German Federal Statistical Office (Destatis) and observed extremely long load times in Microsoft Excel and crashes due to out-of-memory exceptions in software packages used for reading Excel files. Our analysis revealed that these files contain format definitions invisible to the user, which occupy orders of magnitude more storage space than the actual data.

While some existing tools attempt to address this issue, they often fail to fully resolve the problem or introduce new issues, such as altering the visual appearance of spreadsheets. In this paper, we present a comprehensive analysis of the root causes of Excel file bloat, documenting issues that have been partially addressed by existing tools but remained undocumented, and uncovering additional issues not resolved by these tools. We also analyze why these tools impact the visual integrity of Excel files.

We developed an open-source tool, Excel Shrink Analyser, welches wir benutzt haben um herrauszufinden, wodurch die der file bloat in exceldateien entsteht. Auf Grundlage der Ergebnisse wussten wir nach welchen Dateiinhalten wir suchen mussten und haben uns das ziel gesetzt die ein performanceoptimiertes commandozeilenprogramm zu entwickeln, welches dieselben bereinigungen vornimmt aber dabei deutlich schneller ist um so eine geschwindigkeit zu erreichen, die es sinnvoll macht die datei immer erst mit excel Shrink zu bereinigen, bevor man sie mit dem gewünschten Programm oder der gewünschten bibliothekten öffnet, weil die ausführungszeit durch das vorherige bereinigen und das anschließende öffnen durch die bibliothek oder programm schneller ist, als die Excel datei direkt mit dem gewünschten programm oder der gewünschten bibliothek zu öffnen. Wir konnten erfolgreich das programm so optimieren, dass es Excel Dateien öffnen, bereinigen und wieder speichern kann und dabei doppelt so schnell ist, als Microsoft excel es schafft die datei zu öffnen.

Optional: Wir haben auch its graphical user interface counterpart, Excel Shrink UI entwickelt, which effectively address both the known and previously unresolved root causes of Excel file bloat without compromising the visual appearance of the spreadsheets. Excel Shrink UI enables users to optimize their Excel files without the need for programming knowledge or command-line tools. To evaluate our tools, we downloaded all Excel files from Destatis and optimized them using Excel Shrink. Our results show that our tools can reduce file sizes by up to 99.99% without affecting the visual integrity of the files.

**PVLDB Reference Format:**
Alexander Aue-Johr and Hardy Pundt. Excel Shrink: An Open-Source Tool for Reducing File Bloat in Excel Spreadsheets. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

## 1 INTRODUCTION

Excel spreadsheets are integral to a wide range of domains—including finance, statistics, business, and research—due to their flexibility in managing tabular data, executing complex calculations, automating workflows via macros, and visualizing information using charts and graphs [12, 37, 38]. However, as these spreadsheets grow in size and complexity, they often suffer from performance bottlenecks, file bloat, and increased processing delays [7, 9, 44, 45].

Unlike simpler formats such as CSV, Excel files leverage the Office Open XML architecture introduced in Excel 2007 [18]. Under this format, data, formatting, shared strings, and other metadata are stored as a collection of XML files, all compressed into a single ZIP archive. This design supports advanced features, such as separating data from formatting—e.g., centralizing repeated strings in a shared string table [14], or storing fonts, cell styles, and colors in dedicated XML files. While this architecture enables functionality beyond what simpler formats provide, it also creates opportunities for hidden inefficiencies to emerge.

Die Herrausforderungen mit dem filebloat in excel Dateien sind: Excel benötigt einige Zeit um diese Dateien zu öffnen. In excel veringert sich die anzeigegeschwindigkeit auf bloated arbeitsblättern, vor allem beim scrollen. Bilbiotheken, die erlauben excel dateien auszulesen, und in python, R oder anderen Sprachen zu verarbeiten, öffnen diese Dateien ebenso sehr langsam und manchmal stürzen sie sogar durch den file bloat ab, da sie gängige XMl parsing technologien wie einen dom parser nutzen, diese auch mit gewöhnlichen Excel Dateien gut klar kommen, da dort die xml dateien der arbeitsblätter klein genug dafür sind, aber genau diese technologien wie ein domparser können mit diesen arbeitsblättern die an filebloat leiden nicht umgehen, sie sind nicht für enorm


Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

große daten optimiert und stürzen infolgedessen durch outofmemory exceptions ab. Manche Programme wie Texteditoren wie IntelliJ verweigern sogar die vollständige anzeige dieser dateien und zeigen nur den Anfang der Datei. Andere Texteditoren wie visual studio code, öffnen diese Dateien zwar in machen fällen, manchmal aber erst nachdem man den Prompt zum keep waiting oder abbrechen ein bis mehrere male geklickt hat oder es stürzt beim versuch die arbeitsblatt xml Datei zu öffnen ab. Wenn visual studio code doch in der lage ist, eine sehr arbeitsblatt xml Datei zu öffnen, dann erlaubt es aber keine gängigen funktionen die das formatieren des XML codes. Ausgerechnet dies wäre aber hilfreich, om die XML struktur anzusehen um herrauszufinden, wo der bloat in der Datei liegt um ihn ggf manuell zu entfernen.

## 2 BACKGROUND

### 2.1 Excel's Internal Storage Mechanisms and Resulting Inefficiencies

The ZIP-based compression of XML data efficiently reduces file size by exploiting repetitive tags and structures [16]. Yet, this approach can mask significant redundancies. Over time, spreadsheets can accumulate unnecessary formatting or metadata: for instance, entire rows or columns may be formatted even though only a subset of cells contain data [6, 15]. Similarly, duplicating a sheet transfers all formatting, styles, and hidden elements, even when most of them are not needed [21]. Such practices lead to inflated files, slower load times, and degraded performance [21, 39]—issues that become especially problematic in large-scale enterprise environments or when dealing with extensive datasets, such as those provided by the German Federal Statistical Office (Destatis).

## 3 RELATED WORK

A substantial body of research in top-tier venues (VLDB, SIGMOD, ICDE) has explored techniques to *improve performance* on large or complex workbooks. There have also been several efforts to reduce *Excel file size*, with various online resources discussing this topic. We briefly review key findings below, focusing on (i) methods for minimizing file size, (ii) approaches to accelerating computation and user interactions for large spreadsheets, (iii) optimizations for parsing the XML-based `.xlsx` format, and (iv) highlights of Aditya Parameswaran's contributions in bridging spreadsheet and database technologies.

**Minimizing Excel File Size and Bloat.** Microsoft's transition from the binary `.xls` format to `.xlsx` (Office Open XML) introduced compression and a shared string table to reduce repeated data [17, 26]. Further reduction is achieved in the `.xlsb` (binary) format, which stores data in a compact binary container at the cost of certain compatibility constraints [21, 23].

However, there is a lack of peer-reviewed studies specifically quantifying how "bloat" (whitespace values, invisible formatting, rows and column definitions outside of the visible content) affects Excel file size. Instead, most advice on the subject comes from practitioners sharing their experiences. For example, Microsoft's support article on cleaning excess cell formatting provides practical tips for reducing file size and improving performance [24]. Additionally, discussions on forums such as MrExcel illustrate that users have long noticed dramatic size reductions after clearing unused formatting or trimming the "used range" [36]. Popular Excel blogs from UpSlide [47] and The Bricks [46] further detail these methods based on accumulated user experience rather than formal experimentation.

Several proprietary tools have been developed to address Excel bloat automatically. Notable examples include UpSlide's Automation Software, which offers a one-click "Smart Clean" feature [47]; NXPowerLite Desktop [27]; Sobolsoft's Excel File Size Reduce Software [43]; and online services such as WeCompress [48]. Other tools, such as Kernel for Excel Repair [19], have also been mentioned in user communities for their file compression capabilities.

Excel has also developed two tools to assist users. One such tool is **Excel Inquire**, an add-in introduced with Excel 2013 that provides a range of auditing features—including Workbook Analysis, Relationship Diagrams, and the Clean Excess Cell Formatting command. O'Beirne discusses this in the proceedings of the EuSpRIG 2013 Conference on "Spreadsheet Risk Management" [28], noting that the *Clean Excess Cell Formatting* command removes excess formatting but does not indicate beforehand what will be cleared, nor does it report afterwards what was removed.

More recently, Microsoft introduced the **Excel Performance Tab**, a built-in feature designed to monitor workbook performance by identifying inefficiencies and offering cleaning recommendations. However, the Performance Tab has received negative user feedback; several users report that its forced "Check Performance" pop-up and the associated reformatting remove intentional formatting and disrupt workflows [4, 25] and also Excel users report theat their pixel art is deleted [4] while the Microsoft Support Seite even mentions a warning that pixel art will be removed by the tool [25].

In contrast, our paper provides a comprehensive scientific analysis of the root causes of Excel file bloat in XML-based workbooks. To address this gap, we introduce *Excel Shrink*—a free, open-source command-line tool designed for automation pipelines. Unlike proprietary tools that are only accessible through a graphical user interface and sometimes remove too many elements (thereby altering the visual appearance) or too few (leaving the file size large), Excel Shrink selectively reduces file size by eliminating only those elements that do not affect the workbook's visual integrity.

**Performance in Handling Large Spreadsheets.** Rahman et al. [39] conducted a comprehensive study comparing Excel, LibreOffice Calc, and Google Sheets on sizable datasets. They revealed that despite nominal row limits of one million, Excel's GUI would often freeze with mere tens of thousands of rows or extended formula usage. Similar findings appear in other benchmarking efforts [38], highlighting the lack of query-planning or indexing in typical spreadsheet software. Addressing this, "DataSpread" [7] (VLDB 2015) and "Hillview" [11] (PVLDB 2019) offload data management to database engines or distributed clusters, respectively, while retaining a spreadsheet-like interface. These architectures stretch the notion of a "spreadsheet" to hundreds of millions (or billions) of rows by introducing parallelization, indexing structures, or incremental computation. In a different line of work, Bendre et al. [10] proposed an *anti-freeze* model for formula recalculation, allowing asynchronous updates so the user interface does not lock while large spreadsheets recompute.

**Efficient Excel XML Processing.** As modern Excel files (`.xlsx`) are essentially XML (SpreadsheetML) within a ZIP container, *parsing* can become a bottleneck for large sheets. Classical DOM-based XML parsing can exhaust memory on multi-hundred-thousand-row spreadsheets, whereas SAX-based streaming, though more memory-friendly, suffers from high CPU overhead through numerous event callbacks [20]. A hybrid parser that combines DOM and SAX parsing, along with regular expressions for extracting the relevant file contents (e.g. `openxlsx` in R), can still be slow and memory-intensive under worst-case scenarios. By contrast, parsers relying exclusively on DOM parsing (e.g. `readxl` in R) may achieve faster runtime performance but often at the cost of higher memory usage. For instance, the memory consumption of `readxl` can exceed ten times the size of the uncompressed worksheet [20]. To address this, Henze et al. [20] introduced *SheetReader*, a specialized streaming parser that integrates decompression and XML scanning, prunes unneeded formatting metadata early, and parallelizes chunk-level parsing. This yields dramatically lower memory footprints (often 40× less) and faster loads (3× or more) compared to standard libraries.

**Comparisons of Parsing Approaches and Tools.** These studies collectively show a trade-off between *DOM-based* simplicity (at the cost of high memory usage) and fully *SAX-based* pipelines (often incurring higher CPU overhead). Hybrid or domain-specific solutions—like regex-based extraction or streaming chunk processors that skip unneeded style markup—strike a middle ground [20, 40]. Meanwhile, broad tool evaluations (Microsoft Excel vs. Google Sheets vs. LibreOffice Calc) confirm that no mainstream spreadsheet program handles truly large data sets efficiently out of the box [39].

**Parameswaran's Contributions: Bridging Spreadsheets and Databases.** Aditya Parameswaran and collaborators have extensively researched integrating spreadsheets with database backends to achieve both usability and scalability [7, 8]. *DataSpread* preserves the familiar Excel-like interface while relying on a relational DBMS for storage and large-scale computations, enabling interactive performance even at billions of cells. This approach can be seen as bypassing Excel's internal storage and recalculation limits by adopting database features (indexing, query optimization, concurrency control). In related work on "anti-freeze" recalculation, Parameswaran's group explored asynchronous formula evaluation with incremental updates, preventing interface lock-ups for large or complex dependency graphs [10]. Additionally, they highlight practical tactics like saving spreadsheets as binary (`.xlsb`) instead of `.xlsx` to mitigate bloat, limiting volatile formulas (e.g. `OFFSET`), and offloading heavy operations to databases where possible [21]. All these strategies offer evidence that a combination of advanced parsing, background recalculation, and robust data management techniques can significantly enhance performance for real-world Excel workloads.

*Summary.* From straightforward file-format optimizations (`.xlsb` compression) to *architectural* overhauls (DataSpread, Budiu2019Hillview) and specialized parsers (SheetReader), the research consensus is that Excel's native engine and file format alone do not scale gracefully. Modern solutions increasingly combine streaming-based XML handling, database or distributed architectures, and asynchronous computation to push spreadsheet interactivity far beyond Excel's historical limits. These insights strongly motivate the need for further improvements in `.xlsx` bloat reduction and high-performance parsing directly addressed by our proposed approach in this paper.

# 4 INTRODUCTION

**Research Challenge:** Standard XML-based solutions—such as DOM or SAX parsers—become prohibitively slow or run out of memory on these bloated spreadsheets. Our central question is: *"Can we remove spreadsheet bloat without fully decoding each XML file, while still preserving the user's visible layout?"*

**Contributions:**

- We identify key obstacles to high-performance Excel debloating, including hierarchical XML subfiles, hidden formatting, and partial sheets with large row or column definitions.
- We propose a *byte-stream-based*, *multiprocessing* approach that leverages dynamic regular expressions to remove invisible content *in flight*, guided by minimal reading of critical metadata such as `styles.xml` or `sharedStrings.xml`.
- We show that this method can outperform not only typical DOM or SAX-based solutions but even Excel's built-in tools like the Inquire Add-in or *Check Performance*—in many cases completing the cleaning *faster* than the time Excel needs just to open the file.
- We evaluate on over 1,200 real-world Excel files from the German Federal Statistical Office, achieving up to 99.99% reduction in file size with intact layout and style.

Our results have broad implications for data management systems that face large, unwieldy XML-based archives. By highlighting and tackling the difficulties of streaming modifications on nested, compressed data, we aim to enrich the conversation about advanced transformations for file-based data exchange.

# 5 BACKGROUND

# 6 METHODOLOGY2

# 7 METHODOLOGY

*On Streaming Performance vs. Reading the Entire File.* Although our two parsing modes were initially presented as a trade-off between runtime performance and memory usage, we often observe *faster* end-to-end performance when using chunk-based streaming. One key reason is that modern `.xlsx` files are ZIP archives, and Python's `zipfile` module defers the actual decompression and I/O until the stream is read. In other words, *the ZIP is not fully extracted the moment we open the archive*; we only pay the extraction cost when our parser requests bytes from the stream. By reading and processing 8 KiB chunks directly from the compressed archive (rather than decompressing the entire file into memory first), we begin parsing sooner and discard unneeded data immediately. This pipelined approach can outpace a naive load-then-parse strategy because it interleaves:

**Algorithm 1** Processing and Cleaning Excel Files in Place

**Input:**
`InP` ← Excel file path
`OutP` ← Output path

`InArc` ← Open the ZIP archive `InP` for reading
`OutArc` ← Create a new ZIP archive for writing
`ShStr` ← Extract Shared strings from `sharedStrings.xml`
`Sty` ← Extract Styles from `styles.xml`
`SF` ← List of all sheet XML files in `Arc`
`NSF` ← List of non-sheet files in `Arc`

`wvp` ← dynamicly create pattern based on `ShStr` to identify cell values that are whitespace
`icp` ← dynamicly create pattern based on `Sty` to identify cells that are invisible
`sdcp` ← dynamicly create pattern based on `Sty` to identify cells their styles depend on the style of their column and or row style

**for** each sheet file `sf` in `SF` in parallel **do**
  Clean sheet file `sf` streambased using `wvp`, `icp`, `sdcp` and `Sty`
  Write cleaned sheet back to ZIP archive `OutArc`
**end for**

**for** each non-sheet file `nsf` in `NSF` **do**
  **if** `nsf` is `workbook.xml` **then**
    Clean workbook metadata with DOM
    Write cleaned workbook to ZIP archive `OutArc`
  **else**
    Copy file non-sheet file `nsf` unchanged to ZIP archive `OutArc`
  **end if**
**end for**
**Output:** Cleaned Excel file

(1) *I/O and decompression*: The ZIP metadata is read almost instantly, but full decompression proceeds only as our parser requests chunks.
(2) *Selective parsing and discarding*: Unwanted XML regions (e.g. empty rows) are dropped on the fly, reducing downstream overhead.

Hence, even though we initially motivated the streaming parser primarily for memory-efficiency (particularly on very large sheets), we observe beneficial performance side-effects in practice—the parser can start its work earlier, overlap I/O with computation, and avoid holding the entire raw XML in memory before any processing takes place.

*Multi-Threaded Streaming Entry Point.* 1 depicts how we open the ZIP archive and acquire a chunk-based stream for a particular worksheet. We first read small metadata (such as `item.file_size`), but the archive data itself is decompressed only when `sheet_stream.read()` is called inside `clean_sheet_stream(,. . . )`. There, the state machine operates over 8 KiB chunks, incrementally scanning for XML boundaries and discarding irrelevant sections immediately. This design avoids any need to load the entire worksheet file at once

**Algorithm 2** Streaming-Based Cleaning of Excel Sheet Data

**Input:**
`SheetS` ← Sheet XML stream
`CS` ← Chunk size for streaming
`wvp` ← Pattern for whitespace values
`icp` ← Pattern for invisible cells
`sdcp` ← Pattern for cells dependent on row/column styles
`Sty` ← Extracted style definitions

**Initialize:**
`Buf` ← Empty buffer
`State` ← BEFORE_COLS_NODE
`MaxRow` ← 0
`MaxCol` ← 0

**Processing Sheet Stream**
**while** `State` ≠ END_OF_FILE **do**
  Read chunk from `SheetS` into `Buf`

  **if** `State` is BEFORE_COLS_NODE **then**
    Extract and store chunks until `<cols>` or `<sheetData>`
    **if** `<cols>` is found **then**
      `State` ← IN_COLS_NODE
    **else if** `<sheetData>` is found **then**
      `State` ← IN_SHEET_DATA_NODE
    **end if**
  **else if** `State` is IN_COLS_NODE **then**
    Parse column definitions and store column chunks
    Search for closing `</cols>`
    **if** `</cols>` is found **then**
      `State` ← BETWEEN_COLS_AND_SHEET_DATA_NODE
    **end if**
  **else if** `State` is BETWEEN_COLS_AND_SHEET_DATA_NODE **then**
    Extract and store chunks until `SheetData` is found
    **if** `<sheetData>` is found **then**
      `State` ← IN_SHEET_DATA_NODE
    **end if**
  **else if** `State` is IN_SHEET_DATA_NODE **then**
    Remove unnecessary empty/invisible cells using patterns
    Extract cell references to update `MaxRow` and `MaxCol`
    Search for `</sheetData>`
    **if** `</sheetData>` is found **then**
      `State` ← AFTER_SHEET_DATA_NODE
    **end if**
  **else if** `State` is AFTER_SHEET_DATA_NODE **then**
    Extract and store remaining chunks
    If buffer is empty, `State` ← END_OF_FILE
  **end if**
**end while**

**Post-Processing**
Extract and update `MaxRow` and `MaxCol` from merge cell definitions
Remove rows exceeding `MaxRow` from `SheetData`
Merge extraneous columns beyond `MaxCol`

**Output:** Cleaned sheet XML

**Algorithm 3** Processing and Cleaning Excel Sheet Data

**Input:** File path F, sheet item I, sheet mapping M, pattern for whitespace cells P, styles S
**Initialize:** Sheet number N extracted from I, Sheet name L retrieved from M
Create hybrid collections C for column definitions and style mappings
{— Extract and Process Sheet Data —}
Open Excel file as a zip archive
Retrieve the original sheet size from metadata
Open the XML sheet stream
Extract style information such as font-only styles and apply-fill/border styles
Construct regular expressions for detecting empty and styled cells
{— Cleaning Process —}
Apply regex patterns to remove empty cells with font styles
Identify cells with applied fill or border and evaluate their necessity
Remove unnecessary empty rows and whitespace-only cells
Process column definitions and optimize column storage
Reconstruct cleaned XML data
{— Compute Reduction Metrics —}
Compute cleaned sheet size
Compute reduction in bytes and percentage
Print log message with sheet name and reduction statistics
**Output:** Processed sheet item, sheet name, cleaned XML data

**Listing 1: Pseudocode Illustrating the Streaming Entry Point**

```
def process_sheet_data_mp(args):
    file_path, item, sheet_mapping, ...
    sheet_num =
        extract_sheet_number(item.filename)
    sheet_name =
        sheet_mapping.get(sheet_num,
        "Unknown")
    print(f"Processing Sheet {sheet_name}
        ({item.filename})")

    with zipfile.ZipFile(file_path, 'r') as
        z:
        original_size = item.file_size
        with z.open(item.filename) as
            sheet_stream:
            # [ Construct regex patterns,
                etc. ]
            clean_xml_result =
                clean_sheet_stream(
                sheet_stream, 8192,
                    pattern_empty_cells, ...
            )
    # [ Summarize final size and return ]
```

and typically yields better runtime performance in addition to its memory advantages.

## 7.1 Methodology (High-Level State-Machine Overview)

We implement our spreadsheet parser as a streaming-based state machine, where each chunk of XML data is processed incrementally to minimise memory consumption and to permit early discarding of unwanted parts of the file. Both our "consecutive" and "interleaved" parsing approaches, which trade off runtime performance versus memory usage, use this same underlying state machine. In the description below, we focus specifically on how our parser transitions between states (i.e., how each state-changing function decides to advance to a new state), deferring the lower-level details of what happens *within* each state until later sections.

*States.* We define six primary states in our parser:

- `BEFORE_COLS_NODE`: The parser has not yet encountered the `<cols>` element. It searches for either `<cols>` or `<sheetData>` in the stream.
- `IN_COLS_NODE`: The parser has found `<cols>` and is now processing column definitions.
- `BETWEEN_COLS_AND_SHEET_DATA_NODE`: The parser has finished reading (or did not find) the `<cols>` node and looks for the first `<sheetData>` element.
- `IN_SHEET_DATA_NODE`: The parser is processing the main worksheet data, including row and cell elements.
- `AFTER_SHEET_DATA_NODE`: The parser has found the closing `</sheetData>` and is gathering any subsequent chunks, typically until the end of the file.
- `END_OF_FILE`: The parser has reached the end of the stream. No further parsing is required.

*7.1.1 Ensuring "Clean Cuts" within the Streaming Buffer.* Our parser reads the raw XML data incrementally into a working *buffer* and ensures that each XML node boundary is cleanly preserved, so that no node is split across two chunks. Each state-transition function handles this in a slightly different way, as described below. However, the shared principle is that the parser only performs a "cut" in the buffer if it has already read enough bytes to match one of the relevant XML tags (e.g., `<cols>` or `</sheetData>`). If none of the looked-for tags appear yet, *no* cut is made: the function returns without changing the buffer, so the parser simply appends the next chunk to the existing buffer and tries again.

*1.* `collect_chunks_until_cols_sheet_data_or_end_of_stream`. While the parser is in `BEFORE_COLS_NODE`, `BETWEEN_COLS_AND_SHEET_DATA_NODE` or `AFTER_SHEET_DATA_NODE`, this function tries to find specific tag boundaries (e.g., `<cols>`, `<sheetData>`, or `</cols>`). When it detects such a tag at position `i` in the buffer:

(1) It *cuts* the buffer at `i`, meaning it splits the buffer into two segments: *(1)* the portion from the start of the buffer through the end of that tag (the "clean cut"), and *(2)* whatever bytes remain *after* that tag.

(2) It returns the first segment as chunk_to_add (which the parser immediately outputs or processes for the current state).
(3) The second segment becomes the *updated buffer*, ensuring that the relevant XML node boundary is never cut in half.

If the function *fails* to find the tag it is looking for, it returns None for both the new chunk and the new state; *no* modification is done to the buffer. This design permits the next incoming chunk of data to be appended to the unchanged buffer in the following iteration, eventually providing enough content to match the desired tag boundary.

*2.* collect_cols_chunks_and_parse_cols. When the parser is in IN_COLS_NODE, this function specifically searches for </cols> or, failing that, for one or more <col .../> elements. On detecting </cols>, it cuts the buffer right *before* that closing tag and transitions to BETWEEN_COLS_AND_SHEET_DATA_NODE. If the function does not find the closing tag but does discover complete <col .../> elements, it consumes them (again as a clean cut) and leaves the parser in IN_COLS_NODE to continue. As before, if no relevant match is found at all, the buffer is unaltered so that subsequent chunks can be appended for a future match attempt.

*3.* collect_only_visible_row_and_cell_chunks_and_calculate_sheet_dimensions. When parsing the main sheet data in IN_SHEET_DATA_NODE, we similarly look for </sheetData> or row boundaries. We locate the boundary of a row (either a self-closing form, <row .../>, or an open–close form, <row ...>...</row>). If </sheetData> is found, the function cuts the buffer at that tag and transitions the parser to AFTER_SHEET_DATA_NODE. If no row boundary or closing tag is yet fully contained in the buffer, it remains unmodified so that the next chunk from the file can complete the row or sheetData boundary.

*Buffer Updates and State Progress.* By constraining all parsing decisions to these well-defined cuts, the parser never splits an XML node across separate buffers. Each function either (1) makes a precise cut on the matching tag boundary (and updates the parser state accordingly), or (2) leaves the buffer untouched if no boundary is found, ensuring the remainder of the stream can be appended in the subsequent iteration. This incremental procedure allows the parser to operate in a memory-efficient streaming manner while guaranteeing well-formed XML segments in each extracted chunk.

Below, we summarise each function's role in *advancing* the parser from one state to another. (Their internal logic for extracting or removing specific XML chunks is discussed later.)

collect_chunks_until_cols_sheet_data_or_end_of_stream. This function is invoked when the parser is in one of these three states:

- BEFORE_COLS_NODE: It scans the incoming buffer for <cols>. If that appears, the parser transitions to IN_COLS_NODE. Otherwise, it searches for <sheetData> and, if found, transitions to IN_SHEET_DATA_NODE. If neither tag is detected in the current chunk, the parser remains in BEFORE_COLS_NODE and awaits more data.
- BETWEEN_COLS_AND_SHEET_DATA_NODE: It behaves similarly, except the check for <cols> is no longer relevant. Here, the parser only looks for <sheetData> to transition to IN_SHEET_DATA_NODE.
- AFTER_SHEET_DATA_NODE: In this state, the function no longer searches for <cols> or <sheetData>. Instead, it appends any remaining data to the final output and, if the buffer empties, transitions to END_OF_FILE.

collect_cols_chunks_and_parse_cols. This function operates exclusively in IN_COLS_NODE. It looks for the closing </cols> tag:

- If </cols> is found, the parser transitions to BETWEEN_COLS_AND_SHEET_DA
- If it does not find the closing tag but detects one or more <col .../> elements, it incorporates them but remains in IN_COLS_NODE until eventually encountering </cols> or the end of stream.

collect_only_visible_row_and_cell_chunks_and_calculate_sheet_dir
When in IN_SHEET_DATA_NODE, this function looks for the next row element or the closing </sheetData> tag:

- If </sheetData> appears, the parser transitions to AFTER_SHEET_DATA_NOI
- Otherwise, the function gathers row/cell chunks but remains in IN_SHEET_DATA_NODE until the closing tag is encountered in a subsequent chunk.

remove_rows_above_max *and* merge_extraneous_columns. These post-processing routines do not themselves trigger new state transitions. Instead, they perform final cleanup after exiting IN_SHEET_DATA_NODE, typically leaving the parser in AFTER_SHEET_DATA_NODE (or END_OF_FILE if no data remains).

*Putting It All Together.* By orchestrating these transitions, the parser incrementally identifies the <cols> segment (if present) and the main <sheetData> block. Once </sheetData> is found, it switches to a final phase to read any trailing data without further parsing. When no more data remains, it enters END_OF_FILE. Later sections detail how, *within* each state, we selectively discard or retain rows, cells, and styles, ultimately producing a more compact file representation.

*When in* IN_COLS_NODE: collect_cols_chunks_and_parse_cols. While the parser is in the IN_COLS_NODE state, it invokes collect_cols_chunks_a to consume as many <col .../> elements as possible—and if it encounters </cols>, it completes this stage and transitions the state machine. The function proceeds as follows:

(1) **Search for </cols>.**
First, the function looks in the buffer for the exact substring </cols>. If it *finds* this closing tag at position index_closing, it *cuts* the buffer *just before* that tag:
   - chunk_to_add is everything from the start of buffer up to (but not including) </cols>.
   - updated_buffer is the remainder, starting exactly at the </cols> sequence (so subsequent parsing will see that closing tag).
   - It sets new_state to BETWEEN_COLS_AND_SHEET_DATA_NODE, indicating that the <cols> section has finished.
   - parse_cols(chunk_to_add, styles) is called to convert any <col .../> tags within that chunk into ColumnDefinition objects, which are then returned as parsed_cols.

(2) **If no </cols>, look for <col .../> elements.**
If </cols> is not present in the buffer, the function next scans for one or more <col .../> matches. If it finds at least one:
- It identifies the *last occurrence* of such a match (i.e. the rightmost <col .../> in the current buffer).
- It cuts the buffer immediately *after* that last match, so that chunk_to_add includes all <col .../> tags up to that position.
- updated_buffer is what remains after that final match.
- new_state is set to None, which means the parser stays in IN_COLS_NODE (because we have not yet seen </cols>). In subsequent iterations, the parser will continue calling this function to look for more columns or the closing tag.
- parse_cols(chunk_to_add, styles) parses the retrieved <col .../> elements into ColumnDefinition objects.

(3) **If neither is found, return unchanged buffer.**
If the function fails to find either </cols> or any <col .../> elements, it means the buffer does not yet contain a complete column definition or closing-tag boundary. In this scenario, the function returns:
- None for new_state and chunk_to_add,
- The *unchanged* buffer for updated_buffer,
- None for parsed_cols.

Thus, *no* cut is made; the parser appends the next chunk of XML data to the same buffer, giving it a chance to find these tags later once more bytes are read.

*When in* IN_SHEET_DATA_NODE*:* collect_only_visible_row_and_cell_chunks_and_calculate_sheet_dimensions.
Once the parser reaches the main <sheetData> region, it invokes collect_only_visible_row_and_cell_chunks_and_calculate_sheet_dimensions to process rows, remove invisible/empty cells, and (if the closing </sheetData> is found) transition out of this state.

(1) **Search for the last occurrence of any row or the </sheetData> tag.**
The function compiles a regex combined_pattern that matches either:
- The closing tag </sheetData>, or
- A row element (self-closing <row .../> or open–close <row ...> ... </row>).

It scans *backwards* (in effect, by locating the last such match in the buffer), ensuring the parser consumes as many complete row elements as possible without slicing any row tag in half.

(2) **Make a clean cut if a match is found.**
If the function finds one of these patterns, it cuts the buffer at the end of that match:
- chunk_to_add is everything up to and *including* that matched boundary.
- updated_buffer is what remains afterward.
- If the matched tag was </sheetData>, the new state becomes AFTER_SHEET_DATA_NODE. Otherwise (<row ...> match), new_state is None, indicating the parser remains in IN_SHEET_DATA_NODE for possible further rows.

(3) **Remove invisible/empty cells from the retrieved chunk.**
The function then calls the local helper collect_only_visible_row_and_
which applies regex-based filters to:
- Remove fully empty cells and whitespace-only cells,
- Discard cells whose styling indicates hidden or irrelevant content,
- Potentially remove entire rows if they are empty or invisible.

This subfunction also handles a corner case where a row has a style attribute different from certain cells inside it, removing those mismatched cells.

(4) **Update row/column bounds.**
In the cleaned-up chunk_to_add, it searches for all <c ... r="REF"> references. For each cell reference REF, the parser calls coordinate_to_indices_bytes to extract its (column, row) indices. The parser updates last_max_col and last_max_row accordingly, keeping track of the furthest row and column seen so far.

(5) **If no match is found, preserve the buffer.**
If the function does *not* find any row boundary or </sheetData>, it returns:
- None for the new state and the chunk,
- The *unchanged* buffer (so additional data may arrive later),
- The current sheet_dimensions (unused, passed through) and last_max_row/col.

This deferral ensures that partial row data is carried into the next iteration until a full row or the closing tag appears.

By combining these steps, the parser incrementally consumes only complete rows or the final </sheetData> boundary, cleaning out unused cells as it goes, then updates the sheet's dimension bounds before returning to stream further data.

*7.1.2 ColumnRangeMap Data Structure.* In order to efficiently determine the style (or other properties) of a given column index (e.g. by performing style_by_col[col]), we maintain a *range-based lookup* called ColumnRangeMap. Because <col> definitions in an Excel worksheet can each span a range of columns (min–max attributes), we store them in a data structure that combines:

- exact: A hash map for single-column definitions (those where col_def.min == col_def.max). This allows O(1) lookups for the (often frequent) case of narrow column definitions that affect exactly one column index.
- intervals: A sorted list of multi-column ColumnDefinition objects, each spanning [col_def.min, col_def.max]. Alongside this, we store an array of _interval_starts to allow for efficient binary searching based on a given column index.

*Key Operations.*

**add(col_def)**
When inserting a new ColumnDefinition, if the definition covers exactly one column (i.e. col_def.min == col_def.max), the map places it directly into exact[col_def.min]; otherwise, it appends col_def to the intervals list.

**finalize()**
After all definitions are added, this method sorts intervals

by ascending `min` value. It also builds the `_interval_starts` array so we can binary-search the intervals by their `min` coordinate.

**`__getitem__(col)`**
To retrieve a column style for index `col`, we first check if `col` appears in the `exact` map. If not, we perform a `bisect_right(_interval_starts, col)` to find the relevant interval whose `min` boundary is immediately to the left of (or equal to) `col`. If the matched interval covers `col` (i.e. `col` is between `[cd.min, cd.max]`), we return that `ColumnDefinition`. Otherwise, a `KeyError` is raised.

**`__contains__(col)`**
Analogous to `__getitem__`, but returning a boolean. We check `exact`, and if not found there, test the `interval` that `col` might fall into after a binary search in `_interval_starts`.

**`__len__()`**
Reports the total count of column definitions (both single-column and multi-column ranges) by summing the size of `exact` and `intervals`.

This approach ensures that for either a single column or a range, we can efficiently identify the associated `ColumnDefinition`, while also minimising the overhead of searching large lists if many columns share the same style definition. As soon as the parser completes reading the `<cols>` block, it adds each `ColumnDefinition` to the `ColumnRangeMap`, calls `finalize()`, and subsequently uses `style_by_col[col]` lookups to determine per-column styling throughout the sheet-data parsing phase.

*7.1.3 ColumnRangeMap Data Structure.* In order to efficiently determine the style (or other properties) of a given column index (e.g. by performing `style_by_col[col]`), we maintain a *range-based lookup* called `ColumnRangeMap`. Because `<col>` definitions in an Excel worksheet can each span a range of columns (`min`–`max` attributes), we store them in a data structure that combines:

- `exact`: A hash map for single-column definitions (those where `col_def.min == col_def.max`). This allows `O(1)` lookups for the (often frequent) case of narrow column definitions that affect exactly one column index.
- `intervals`: A sorted list of multi-column `ColumnDefinition` objects, each spanning `[col_def.min, col_def.max]`. Alongside this, we store an array of `_interval_starts` to allow for efficient binary searching based on a given column index.

*Key Operations.*

**`add(col_def)`**
When inserting a new `ColumnDefinition`, if the definition covers exactly one column (i.e. `col_def.min == col_def.max`), the map places it directly into `exact[col_def.min]`; otherwise, it appends `col_def` to the `intervals` list.

**`finalize()`**
After all definitions are added, this method sorts `intervals` by ascending `min` value. It also builds the `_interval_starts` array so we can binary-search the intervals by their `min` coordinate.

**`__getitem__(col)`**
To retrieve a column style for index `col`, we first check if `col` appears in the `exact` map. If not, we perform a `bisect_right(_interval_starts, col)` to find the relevant interval whose `min` boundary is immediately to the left of (or equal to) `col`. If the matched interval covers `col` (i.e. `col` is between `[cd.min, cd.max]`), we return that `ColumnDefinition`. Otherwise, a `KeyError` is raised.

**`__contains__(col)`**
Analogous to `__getitem__`, but returning a boolean. We check `exact`, and if not found there, test the `interval` that `col` might fall into after a binary search in `_interval_starts`.

**`__len__()`**
Reports the total count of column definitions (both single-column and multi-column ranges) by summing the size of `exact` and `intervals`.

This approach ensures that for either a single column or a range, we can efficiently identify the associated `ColumnDefinition`, while also minimising the overhead of searching large lists if many columns share the same style definition. As soon as the parser completes reading the `<cols>` block, it adds each `ColumnDefinition` to the `ColumnRangeMap`, calls `finalize()`, and subsequently uses `style_by_col[col]` lookups to determine per-column styling throughout the sheet-data parsing phase.

Our methodology evolved from an initial DOM-based prototype (*Excel Shrink Analyzer*) to our newer *stream-based approach*:

(1) **Root-Cause Analysis of Bloat.** We first used a DOM parser to profile massive Excel files in memory, labelling the main sources of bloat: hidden rows, empty but styled cells, bloated shared strings, or extraneous columns.
(2) **Justification for Streaming.** Observing frequent out-of-memory issues and poor scaling with existing XML libraries, we hypothesised that *regex-based chunk processing* of the raw bytestream might be possible if we tracked enough context (row/column state, style references) in a small buffer.
(3) **Regex Construction.** Because Excel's XML is hierarchical, naive regex is insufficient. We build a pipeline of multiple, smaller expressions, triggered in stages:
   (a) Identify whitespace-only shared strings, remove referencing cells if no visible style is applied.
   (b) Delete purely empty cells with no border or fill.
   (c) Truncate rows/columns outside the bounding box of the visible data region.
   (d) Prune extraneous `cols` definitions while leaving the final set extended to `XFD` (the maximum Excel column), preserving layout.

## 8 METHODOLOGY3

Our approach addresses a key question in large-scale spreadsheet processing: how can we *stream* data from a partially-compressed XML structure without decompressing the entire file or constructing bulky in-memory representations (e.g. a full DOM tree)? We propose a *chunk-based streaming parser* that operates directly on compressed `.xlsx` data and interleaves I/O, decompression, and selective filtering. While our concrete implementation targets Excel worksheets, the overall technique—state-driven streaming, minimal buffer usage, and early discarding of irrelevant data—is more generally applicable to other large, ZIP-compressed XML formats.

## 8.1 Core Insight: Chunk-Based Streaming for XML

Most off-the-shelf XML parsers assume a fully uncompressed data source. By contrast, Python's `zipfile` module defers decompression until we *explicitly* request more bytes. We exploit that behavior to process the archive in small increments (e.g. 8 KiB at a time) rather than unpacking everything at once. The design confers three main advantages:

(1) **Low memory overhead.** The parser never loads the entire XML tree, thus avoiding excessive DOM creation for large sheets.

(2) **Immediate discard of unused content.** Surplus or hidden rows can be dropped as soon as they appear in the partial XML buffer.

(3) **Faster end-to-end throughput.** Decompression proceeds in tandem with parsing, so we do not spend extra time inflating elements that will be discarded anyway.

## 8.2 Parser Overview and State Machine

Our parser maintains a small, multi-state machine that enforces *clean cuts* at recognized XML boundaries. Concretely:

(1) We request a chunk (e.g. 8 KiB) from the partially decompressed archive and append it to an in-process buffer.

(2) We search the buffer for specific tag boundaries (e.g. `<sheetData>`, `<cols>`, or `</row>`). If found, we split ("cut") the buffer so that one chunk ends precisely at that boundary, returning a self-contained fragment of XML.

(3) We process or discard the extracted fragment immediately, filtering out unneeded data, then update the parser's *state* (e.g. `IN_COLS_NODE`→`BETWEEN_COLS_AND_SHEET_DATA_NODE`) depending on which boundary was encountered.

(4) If no relevant boundary is found, we simply read the next chunk from the archive and keep accumulating data until we can form a well-defined fragment.

*States.* The parser transitions among six states: `BEFORE_COLS_NODE`, `IN_COLS_NODE`, `BETWEEN_COLS_AND_SHEET_DATA_NODE`, `IN_SHEET_DATA_NODE`, `AFTER_SHEET_DATA_NODE`, and `END_OF_FILE`. These reflect the major sections of Excel's XML schema. Each state has its own logic for scanning boundaries and deciding how to split or discard data. Crucially, by preserving node boundaries within each chunk, the parser never straddles a tag across multiple buffers.

## 8.3 Selective Filtering of Rows and Cells

Within the `<sheetData>` region, we apply small regular-expression filters to remove unneeded elements. These include:

(1) **Empty or whitespace-only cells** with no style or border definitions.

(2) **Hidden or unnecessary columns and rows** identified by Excel-specific metadata (`hidden="1"`) or by comparing row/column references to the maximum visible region.

(3) **Redundant definitions** that do not affect the final layout (e.g. extraneous `<col>` elements spanning columns beyond the highest-occupied column).

Because our parser operates incrementally, each chunk can discard such elements on the fly, freeing memory sooner than a load-then-parse approach.

## 8.4 Range-Based Column Style Lookups

Excel often encodes column styles via `<col>` elements with a `min–max` range. We represent these definitions using a *range map*: single-column definitions go into a fast hash map, while multi-column definitions are placed into a sorted interval list. This makes column-level style checks efficient and avoids generating thousands of nearly-identical `<col>` elements.

## 8.5 Methodology Summary and Empirical Results

We designed the parser to:

(1) *Minimise memory usage* by never building a global DOM tree.

(2) *Start parsing sooner*, overlapping I/O, decompression, and row-level filtering.

(3) *Guarantee well-formed partial XML chunks* with no risk of splitting tags across buffers.

In §**??**, we quantify the performance payoff of streaming. Compared to a DOM-based baseline, our approach reduces peak memory usage by up to 70% and often improves speed. The measured gains stem from immediately discarding irrelevant rows, cells, or columns before they inflate memory or cause needless parse overhead.

*Full pseudocode and additional lower-level implementation details (including boundary-handling examples) are available in Appendix A.*

# 9 IMPLEMENTATION

We implemented our streaming approach as a Python tool named *Excel Shrink* (publicly available via GitHub[1]). Below, we highlight key aspects:

*Archive-Wide Streaming.* We open the `.xlsx` (ZIP container) and process each subfile. Non-XML subfiles (images, macros) are passed through unmodified. For each XML subfile, we:

(1) Read in small chunks (e.g. 8–64 KB) and maintain a *rolling buffer* that ends exactly at XML tag boundaries relevant for transformations.

(2) Use a *state machine* (e.g. BeforeColsNode, InColsNode, InSheetDataNode, etc.) to trigger different sets of regex replacements on each chunk.

(3) Patch the chunk in place, removing or editing text nodes, then write out the result to a new ZIP stream.

*Multiprocessing.* To exploit modern multi-core CPUs, we parallelise the processing of large worksheets. Each sheet is streamed in a separate Python process, minimising memory overhead. We rejoin them in the final ZIP seamlessly.

*Metadata Handling.* We partially parse only `styles.xml` and `sharedStrings.xml`, building minimal dictionaries of border/fill references and textual content indices. This small overhead is required to decide if an empty cell is truly invisible or if it has a visually significant style.

*Performance vs. Full Parsing.* SAX-based or DOM-based code can require hundreds of seconds (or fail entirely) on multi-gigabyte expansions of a ".xlsx." In contrast, we keep each chunk under 100 KB, apply local transformations, and forward the rest unparsed.

[1] https://github.com/...

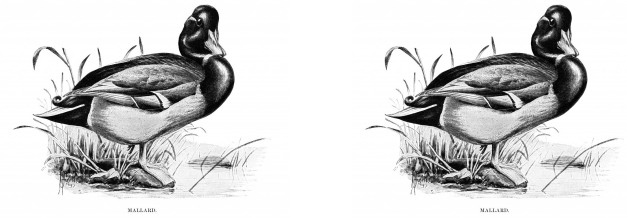


**Figure 1: (Left) Distribution of file size reduction across 1,200 Excel sheets. (Right) Runtime ratio of different approaches. Lower is better.**

## 10 EVALUATION

We tested our approach on 1,200+ real Excel files from the German Federal Statistical Office, covering year-based directories and containing numerous oversize spreadsheets. We also compared performance to:

- **DOM-based approach** using `lxml` in Python
- **SAX-based approach** using `xml.sax`
- **Microsoft Excel 365 Check Performance** (manually invoked)

*File Size Reduction.* Figure 1 (left) shows that 35% of sheets were reduced by over 90%, and 15% of them exceeded 99% reduction. The largest single sheet dropped from 85 MB to 2.76 MB, all while preserving layout and content.

*Runtime Comparison.* Figure 1 (right) compares the wall-clock runtime. We normalise *Excel Shrink*'s runtime to the time Excel needed just to open each file. On average, we completed the shrink *20% faster* than Excel's open time, and in many cases, the total time for "shrink + then open in Excel" was still *shorter* than opening the original file directly in Excel.

### 10.1 Implications and Discussion

Our results underscore that streaming partial XML transformations can surpass naive approaches (DOM/SAX) in both memory and runtime. Moreover, the synergy of chunk-based processing and style metadata ensures that we can preserve layout reliably, avoiding the "broken formatting" problem in many existing bloat-reduction tools.

## 11 CONCLUSION

We introduced a streaming-based, multiprocessing strategy for removing Excel file bloat at scale, demonstrating that it can outperform native Excel loading times. By combining partial XML parsing for style metadata with a pipeline of dynamic regex transformations, we preserve worksheet layout while eliminating up to 99.99% of the storage overhead.

Beyond Excel, our contribution highlights how domain-specific streaming transformations can circumvent memory-intensive parsing in other compressed XML-based systems. Future work includes porting our approach to lower-level languages (C++, Rust) for even higher throughput, and extending the technique to related file formats such as ODS (OpenDocument Spreadsheet). We believe that bridging the gap between domain-specific knowledge (styles, merges, hidden columns) and high-performance streaming offers a promising avenue for *truly large* spreadsheet management in the broader data management community.

## REFERENCES


[1] [n.d.]. Destatis XLSX Files Shrunk with Excel Shrink. https://anonymous.4open.science/r/shrinked_destatis_xlsx_files-4E27/. Accessed: 2024-12-07.

[2] [n.d.]. Original Destatis XLSX Files Before Shrinking with Excel Shrink. https://anonymous.4open.science/r/original_destatis_xlsx_files-5B7B/. Accessed: 2024-12-07.

[3] [n.d.]. Results of Excel Shrink on Destatis XLSX Files. https://anonymous.4open.science/r/excel-shrink-results-7468/shrink_results.csv. Accessed: 2024-12-08.

[4] 2024. Excel 2024: Check Performance in Excel. https://www.mrexcel.com/excel-tips/excel-2024-check-performance-in-excel/. Accessed: 2025-02-26.

[5] 2024. Excel Shrink UI - An open-source graphical user interface for Excel Shrink. https://anonymous.4open.science/r/excel-shrink-ui-BE5C/README.md.

[6] Rui Abreu, Jácome Cunha, Joao Paulo Fernandes, Pedro Martins, Alexandre Perez, and Joao Saraiva. 2014. Faultysheet detective: When smells meet fault localization. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 625–628.

[7] Mangesh Bendre, Bofan Sun, Ding Zhang, Xinyan Zhou, Kevin Chen-Chuan Chang, and Aditya G. Parameswaran. 2015. DataSpread: Unifying Databases and Spreadsheets. *PVLDB* 8, 12 (2015), 2000–2003. https://doi.org/10.14778/2824032.2824121

[8] Mangesh Bendre, Vipul Venkataraman, Xinyan Zhou, Kevin Chen-Chuan Chang, and Aditya G. Parameswaran. 2018. Towards a Holistic Integration of Spreadsheets with Databases: A Scalable Storage Engine for Presentational Data Management. In *Proceedings of the 34th IEEE International Conference on Data Engineering (ICDE)*. IEEE, Paris, France, 113–124. https://doi.org/10.1109/ICDE.2018.00020

[9] Mangesh Bendre, Tana Wattanawaroon, Kelly Mack, Kevin Chang, and Aditya Parameswaran. 2019. Anti-freeze for large and complex spreadsheets: Asynchronous formula computation. In *Proceedings of the 2019 International Conference on Management of Data*. 1277–1294.

[10] Mangesh Bendre, Tana Wattanawaroon, Kelly Mack, Kevin Chen-Chuan Chang, and Aditya G. Parameswaran. 2019. Anti-Freeze for Large and Complex Spreadsheets: Asynchronous Formula Computation. In *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data (SIGMOD '19)*. ACM, New York, NY, USA, 1277–1294. https://doi.org/10.1145/3299869.3319876

[11] Mihai Budiu, Parikshit Gopalan, Lalith Suresh, Udi Wieder, Han Kruiger, and Marcos K. Aguilera. 2019. Hillview: A Trillion-Cell Spreadsheet for Big Data. *PVLDB* 12, 11 (2019), 1442–1457. https://doi.org/10.14778/3342263.3342279

[12] George Chalhoub and Advait Sarkar. 2022. "It's freedom to put things where my mind wants": Understanding and improving the user experience of structuring data in spreadsheets. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–24.

[13] Microsoft Corporation. 2024. Workbook Class (DocumentFormat.OpenXml.Spreadsheet) | Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.spreadsheet.workbook?view=openxml-3.0.1. Accessed on October 7, 2024.

[14] Microsoft Corporation. 2024. Working with the shared string table | Microsoft Learn. https://learn.microsoft.com/en-us/office/open-xml/spreadsheet/working-with-the-shared-string-table?tabs=cs. Accessed on October 7, 2024.

[15] Jácome Cunha, Joao Paulo Fernandes, Pedro Martins, Jorge Mendes, and Joao Saraiva. 2012. Smellsheet detective: A tool for detecting bad smells in spreadsheets. In *2012 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 243–244.

[16] Peter Deutsch. 1996. *DEFLATE compressed data format specification version 1.3*. Technical Report.

[17] Ecma International. 2007. *Office Open XML Formats*. Technical Report. Ecma International. https://www.ecma-international.org/wp-content/uploads/Office-Open-XML-White-Paper.pdf Accessed: 2025-02-27.

[18] Ecma International. 2021. *ECMA-376: Office Open XML File Formats* (5th ed.). Technical Report ECMA-376. Ecma International. https://ecma-international.org/publications-and-standards/standards/ecma-376/ ISO/IEC 29500, Technical Committee TC45.

[19] Kernel for Excel Repair. [n.d.]. Kernel for Excel Repair. https://www.reddit.com/r/excel/comments/1awtq8d/how_to_decrease_excel_file_size_and_speed_up/. Accessed: 2025-02-26.

[20] Felix Henze, Haralampos Gavriilidis, Eleni Tzirita Zacharatou, and Volker Markl. 2022. Efficient Specialized Spreadsheet Parsing for Data Science. In *Proceedings of the 24th International Workshop on Design, Optimization, Languages and Analytical Processing of Big Data (DOLAP) (CEUR Workshop Proceedings)*, Vol. 3130. CEUR-WS.org, 41–50. http://ceur-ws.org/Vol-3130/paper5.pdf

[21] Kelly Mack, John Lee, Kevin Chen-Chuan Chang, Karrie Karahalios, and Aditya G. Parameswaran. 2018. Characterizing Scalability Issues in Spreadsheet Software

using Online Forums. In *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems (CHI EA '18)*. ACM, New York, NY, USA, Paper CS04, 1–9. https://doi.org/10.1145/3170427.3174359
[22] Microsoft. 2014. *Open XML SDK 2.5 for Office*. Microsoft Press.
[23] Microsoft Support. [n.d.]. Reduce the file size of your Excel spreadsheets. Microsoft Support Article. https://support.microsoft.com/en-us/office/reduce-the-file-size-of-your-excel-spreadsheets-c4f69e3a-8eea-4e9d-8ded-0ac301192bf9 Saving a workbook as an Excel Binary Workbook (.xlsb) instead of the default Excel Workbook (.xlsx) can significantly reduce the file size.
[24] Microsoft Support. [n.d.]. Reduce the file size of your Excel spreadsheets. https://support.microsoft.com/en-us/office/reduce-the-file-size-of-your-excel-spreadsheets-c4f69e3a-8eea-4e9d-8ded-0ac301192bf9. Accessed: 2025-02-26.
[25] Microsoft Support. n.d.. Cleanup cells in your workbook. https://support.microsoft.com/en-us/office/cleanup-cells-in-your-workbook-edcc579f-b82f-495b-8d31-e786cd11717b. Accessed: 2025-02-26.
[26] Microsoft Support. n.d.. The file size of an Excel workbook may unexpectedly increase exponentially. https://support.microsoft.com/en-us/topic/the-file-size-of-an-excel-workbook-may-unexpectedly-increase-exponentially-b1e8e0ad-94a8-f97f-5adc-2e22798beac9. Accessed: 2025-02-27.
[27] Neuxpower. 2024. NXPowerLite Desktop - File Compressor Software. https://neuxpower.com/nxpowerlite-desktop/. Accessed on October 7, 2024.
[28] Patrick O'Beirne. 2013. Excel 2013 Spreadsheet Inquire. In *Proceedings of the EuSpRIG 2013 Conference "Spreadsheet Risk Management"*. European Spreadsheet Risks Interest Group. http://www.sysmod.com/xltest Accessed: 2025-02-26.
[29] Patrick O'Beirne. 2014. Excel 2013 Spreadsheet Inquire. *arXiv preprint arXiv:1401.7586* (2014).
[30] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2021. Förderprogramme - Landwirtschaftszählung 2020 (Support Programs - Agricultural Census 2020). https://www.destatis.de/DE/Themen/Branchen-Unternehmen/Landwirtschaft-Forstwirtschaft-Fischerei/Landwirtschaftliche-Betriebe/Publikationen/Downloads-Landwirtschaftliche-Betriebe/foerderprogramme-5411206209004.html Accessed: 2024-12-08.
[31] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2022. Verkehrsunfälle - Zeitreihen 2021 (Traffic Accidents - Time Series 2021) (Letzte Ausgabe - berichtsweise eingestellt). https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Verkehrsunfaelle/Publikationen/Downloads-Verkehrsunfaelle/verkehrsunfaelle-zeitreihen-pdf-5462403.html. Accessed on October 7, 2024.
[32] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2022. Wanderungen - Fachserie 1 Reihe 1.2 - 2021 (Migration - Series 1, Series 1.2 - 2021) (Letzte Ausgabe - berichtsweise eingestellt). https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Bevoelkerung/Wanderungen/Publikationen/Downloads-Wanderungen/wanderungen-2010120217004.html. Accessed on October 7, 2024.
[33] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2023. Private Konsumausgaben und Verfügbares Einkommen - 1. Vierteljahr 2023 (Private Consumption Expenditures and Disposable Income - 1st Quarter 2023) (Letzte Ausgabe - berichtsweise eingestellt). https://www.destatis.de/DE/Themen/Wirtschaft/Volkswirtschaftliche-Gesamtrechnungen-Inlandsprodukt/Publikationen/Downloads-Inlandsprodukt/konsumausgaben-pdf-5811109.html. Accessed on October 7, 2024.
[34] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2023. Statistischer Bericht - Rechnungsergebnisse der Kernhaushalte der Gemeinden und Gemeindeverbände 2021 (Statistical Report - Financial Results of the Core Budgets of Municipalities and Municipal Associations 2021). https://www.destatis.de/DE/Themen/Staat/Oeffentliche-Finanzen/Ausgaben-Einnahmen/Publikationen/Downloads-Ausgaben-und-Einnahmen/statistischer-bericht-rechnungsergebnis-kernhaushalt-gemeinden-2140331217005.html. Accessed on October 7, 2024.
[35] Statistisches Bundesamt Destatis (German Federal Statistical Office). 2024. robots.txt. https://www.destatis.de/robots.txt. Accessed on October 7, 2024.
[36] Users on MrExcel. [n.d.]. File size increased & slow after formulas and conditional formatting. https://www.excelforum.com/excel-general/762063-file-size-increased-and-slow-after-formulas-and-conditional-formatting.html. Accessed: 2025-02-26.
[37] Raymond R Panko. 1998. What we know about spreadsheet errors. *Journal of Organizational and End User Computing (JOEUC)* 10, 2 (1998), 15–21.
[38] Sajjadur Rahman, Mangesh Bendre, Yuyang Liu, Shichu Zhu, Zhaoyuan Su, Karrie Karahalios, and Aditya G. Parameswaran. 2021. NOAH: Interactive Spreadsheet Exploration with Dynamic Hierarchical Overviews. *PVLDB* 14, 6 (2021), 970–983. https://doi.org/10.14778/3447689.3447701
[39] Sajjadur Rahman, Kelly Mack, Mangesh Bendre, Ruilin Zhang, Karrie Karahalios, and Aditya G. Parameswaran. 2020. Benchmarking Spreadsheet Systems. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. ACM, New York, NY, USA, 1589–1599. https://doi.org/10.1145/3318464.3389782
[40] Philipp Schauberger and Alexander Walker. 2021. *openxlsx: Read, Write and Edit xlsx Files*. R Foundation for Statistical Computing, Vienna, Austria. https://doi.org/10.32614/CRAN.package.openxlsx R package version 4.2.5.
[41] Prash Shirolkar. 2022. Check Performance in Excel for the Web. https://techcommunity.microsoft.com/t5/microsoft-365-insider-blog/check-performance-in-excel-for-the-web/ba-p/4216095. Microsoft 365 Insider Blog, Accessed: 2024-04-27.
[42] Prash Shirolkar. 2024. Make Your Workbooks More Performant with Check Performance in Excel for Windows. https://techcommunity.microsoft.com/t5/microsoft-365-insider-blog/make-your-workbooks-more-performant-with-check-performance-in/ba-p/4224276 Accessed: 2024-10-18.
[43] Sobolsoft. [n.d.]. Excel File Size Reduce Software. https://www.sobolsoft.com/excelsize/. Accessed: 2025-02-26.
[44] Alaaeddin Swidan, Felienne Hermans, and Ruben Koesoemowidjojo. 2016. Improving the performance of a large scale spreadsheet: a case study. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 1. IEEE, 673–677.
[45] Dixin Tang, Fanchao Chen, Christopher De Leon, Tana Wattanawaroon, Jeaseok Yun, Srinivasan Seshadri, and Aditya G Parameswaran. 2023. Efficient and Compact Spreadsheet Formula Graphs. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2634–2646.
[46] The Bricks. 2025. How to Reduce Excel Sheet Size. https://thebricks.com/resources/guide-how-to-reduce-excel-sheet-size. Accessed: 2025-02-27.
[47] UpSlide. 2024. How To Reduce Excel File Size in 8 Easy Ways. https://upslide.net/blog/reduce-excel-file-size/. Accessed: 2025-02-26.
[48] WeCompress. [n.d.]. Online File Compressor for Excel Files. https://www.youcompress.com/excel/. Accessed: 2025-02-26.

# A EXCEL SHRINK

The objective of *Excel Shrink* is to reduce the size of *Excel* files by removing content that is invisible to the user and likely unintentionally included in the file. This includes empty cells, cells containing only whitespace, cells with non-visible formatting, and unused rows and columns.

## A.1 Deletion of White Spaces

Excel users lack a straightforward method to identify which cells in a worksheet contain only whitespace characters. To highlight these whitespaces, we have marked cells containing exclusively whitespaces in red within this worksheet, which includes 4,121 such instances (see Figure 2) [33].

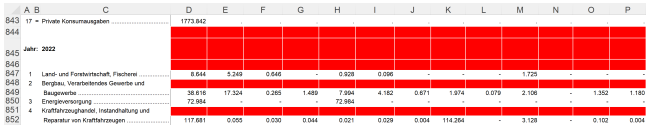


**Figure 2: Cells containing only whitespaces are highlighted in red.**

**Listing 2: Cell `D844` in `sheet28.xml`**

```
<row r="844" spans="1:16" ht="14.1"
    customHeight="1">
  <c r="D844" s="428" t="s">
      <v>102</v>
  </c>
```

To detect these whitespaces, it is insufficient to examine the cell itself, as even a single space is stored by *Excel* in the file for recurring texts, even if the text appears only once in the entire *Excel* file. Cell `D844` appears in the XML as shown in Listing 2.

The row is marked by the `<row>` tag with the attribute `r="844"`, indicating that it is the 844th row. Within the row, the `<c>` tag

represents a cell, identified by the attribute `r="D844"`. The attribute `t="s"` specifies the cell type; in this case, `s` stands for *String* [22].

The cell's value is stored within the `<v>` tag, where the value `102` is specified. This value is interpreted as an index in a string table, referencing a specific text in a separate list defined in the *Excel* file, namely the `sharedStrings.xml` file.

Since *Excel* stores even single spaces in the Shared String Table, merely inspecting the cell's content is insufficient to detect whitespaces. By examining the shared strings, we can identify entries that contain only whitespace characters or are empty, allowing us to determine which cells can be safely deleted. To locate the value, we must search for the `<sst>` node in the `sharedStrings.xml` file, as shown in Listing 3.

**Listing 3: Shared String Table (`sst`) in `sharedStrings.xml`**

```
<sst count="17962" uniqueCount="732">
  <si>
    <t>062</t>
  </si>
```

The provided XML excerpt displays a portion of the Shared String Table (SST) in an *Excel* file [14]. The Shared String Table is a structure that allows for the storage and management of shared strings within an *Excel* workbook. Instead of storing each string separately in every cell, each unique string is stored once in the Shared String Table. This approach saves storage space, as cells then reference these strings via indices. Each entry in this table has an ascending index number, starting at `0` for the first string, `1` for the second, and so forth. The `count` attribute indicates the total number of strings in the table, while `uniqueCount` represents the number of unique strings. In this case, there are `17,962` strings in total, of which `732` are unique. This means that many strings in the table are used multiple times.

Within the SST, there are elements of type `<si>` (Shared Item), each containing a string. In this example, the `<t>` element displays the actual text of the string stored in the table. Here, the string `062` is represented, which can be referenced by an index in a cell.

To find the string corresponding to index `102`, we must look for the 103rd entry (since indexing starts at 0), which is shown in Listing 4.

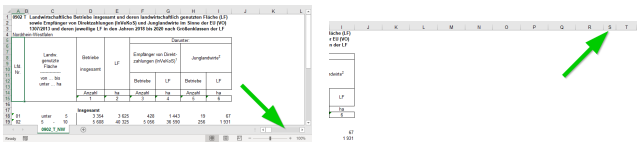
**Listing 4: Empty string at index 102 in `sst`**

```
<si>
  <t xml:space="preserve"></t>
</si>
```

Within the `<si>` element is a `<t>` element containing the actual text of the string. In this specific case, the content of the `<t>` element is empty, indicating that no text is stored. The attribute `xml:space="preserve"` suggests that this represents a whitespace character. Therefore, all cells referencing index `102` for the reusable string can be deleted, as they contain no visible content.

## A.2 Deletion of Empty Columns

The first indication of excessive column definitions can be observed in the horizontal scrollbar when opening the worksheet, as shown in Figure 3a [30, Sheet: `0902_T_NW`]. If the entire data range is already within the visible area and yet the scrollbar thumb is very narrow and small compared to the scrollable area, it indicates that a large number of column definitions are making the worksheet significantly wider than the actual data range. Scrolling into this area, the user may notice that among columns with uniform widths, some columns occasionally have differing widths, as shown in Figure 3b.



**(a) Very wide scrollable area and very narrow scrollbar thumb**

**(b) Custom Column Width for a Column Outside the Data Range**

**Figure 3**

These column definitions are specified in *Excel* within the file `sheetX.xml` under the `cols` node. In our example, the definition of the initial columns is shown in Listing 5.

In this XML excerpt, the `<cols>` element represents the definition of multiple columns in an *Excel* worksheet. Each `<col>` declaration contains specific information about one or more columns. The `min` and `max` attributes specify the range of columns that the `<col>` element covers. For example, `min="1" max="1"` pertains only to column `1`, while `min="5" max="9"` describes columns `5` through `9`. The `width` attribute specifies the width of the columns in *Excel* units. The `customWidth="1"` attribute indicates that the column width is user-defined, meaning it has been manually adjusted rather than using the default value.

The first five `<col>` definitions relate to the data range. However, the last definition, `min="10" max="18"`, pertains to a redundant range. This section defines columns that lie outside the actual data range in the worksheet, specifically columns starting from column J, thereby representing unnecessary or unused definitions.

**Listing 5: Initial column definitions**

```
<cols>
  <col min="1" max="1" width="5.21875"
      style="39" customWidth="1" />
  <col min="2" max="2" width="1.77734375"
      style="39" customWidth="1" />
  <col min="3" max="3" width="20"
      style="39" customWidth="1" />
  <col min="4" max="4" width="11.5546875"
      style="39" customWidth="1" />
  <col min="5" max="9" width="10.5546875"
      style="39" customWidth="1" />
  <col min="10" max="18" width="11.5546875"
      style="54" customWidth="1" />
```

During our investigations, we could not determine how these column definitions originated. We hypothesize that the worksheet was duplicated, and the original worksheet contained this number of columns with the corresponding widths. However, this hypothesis is contradicted by the fact that the number of columns in the files

we examined approaches *Excel*'s maximum limit of 16,384 columns. Figure 4 illustrates such a case [30, Sheet: `0902_T_NW`]. This worksheet contains 2,403 column definitions, and after excluding the five actually used column definitions, 2,398 redundant definitions remain.


**Figure 4: Excessive Column Definitions in a Worksheet**

Columns `WWA` and the partially visible `WWB` correspond to the 16,147th and 16,148th columns, respectively. It seems improbable that an original worksheet would require the utilization of such an extensive number of columns. Further research is required to determine how such patterns emerge in *Excel* files.

The last visible columns in Figure 4 are shown in Listing 6. The first column `min="16146" max="16146"` has a fixed width of `2.21875` and refers to column `WVZ`, which is visible as the column immediately to the left of `WWA` but is too narrow to display more than the letter `W`. The second column `min="16147" max="16147"` defines column `WWA` with a width of `5.21875`. The final definition `min="16148" max="16384"` covers a larger range from column `16148` to `16384`, corresponding to columns `WWB` through `XFD` (the last column in *Excel*). This explains why scrolling all the way to the right via the scrollbar lands on column `WWB`. The penultimate definition specifies where the data range ends, thereby also defining where the scrollable range ends, and the final definition simply assigns a default width to all remaining columns. Surprisingly, these column definitions extend all the way to *Excel*'s maximum limit of 16,384 columns. We have no explanation for why these user-defined column widths appear up to just before the maximum allowable column, but not exactly at it.

To delete these columns, the following steps must be taken:

(1) Identify the actual right boundary of the data range and cell formats.
(2) Determine the first redundant column definition and save the `min` value to use it later.
(3) Delete all redundant column definitions except for the last one.
(4) Overwrite the `min` value of the last column definition with the saved value of the first redundant column definition.

**Listing 6: Final column definitions in the worksheet**

```
<col min="16146" max="16146"
    width="2.21875" style="39"
    customWidth="1" />
<col min="16147" max="16147"
    width="5.21875" style="39"
    customWidth="1" />
<col min="16148" max="16384"
    width="8.77734375" style="39"
    customWidth="1" />
</cols>
```

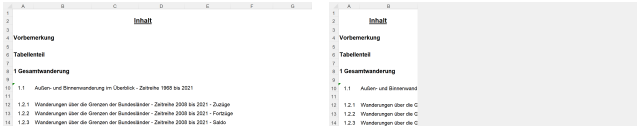
Following these steps is crucial. We experimented with merely deleting the excess columns without ensuring that the last column definition extended to the final possible column (16,384), but this caused *Excel* to mark these worksheets as corrupted and perform certain repairs. Part of these changes included altering the default row height, resulting in some worksheets appearing visually altered, as shown in Figures 5a and 5b. On the left is the original, and on the right is the repaired version by *Excel*. For instance, row 6 has changed in height due to the repair, causing visible shifts in the worksheet. Correctly deleting column definitions while ensuring the last column definition extends to the column limit of 16,384 prevents this error.


**(a) Original worksheet before repair** **(b) Worksheet after repair by *Excel***

**Figure 5: Visual differences due to changes in default row height**

*A.2.1 Hidden Columns.* Removing columns is not recommended if there are hidden columns. In spreadsheets like *Excel*, cells define only where text begins. If the text is longer than the cell, it typically extends over adjacent cells unless those cells also contain text. Some worksheets rely on the assumption that text will overflow into adjacent cells, causing the range of actually used cells to span multiple columns. In such cases, it's important not to delete columns outside the range of cells with values. Doing so could shift hidden columns to the next position, causing text from previous cells to overflow into them. This is depicted in Figures 6a and 6b. Since the texts are only defined in columns `A` and `B`, all cells from column `C` onwards are outside the range of cells with values and are thus considered for deletion. However, column `C` is already hidden, causing the texts to be cut off.

When *Excel Shrink* detects hidden columns, it avoids removing them to ensure that the worksheet's visual representation is not altered.




**(a) Worksheet before column deletion** **(b) Worksheet after column deletion**

**Figure 6: Effect of deleting columns when hidden columns are present**

## A.3 Deletion of Empty Cells

Some of the worksheets we examined contained vast numbers of cells without any content. The most extreme case was a worksheet

with 8,141 cells containing values and 5,237,075 cells without values, meaning that only 0.15% of the stored cells actually had a value [34, Sheet: csv-71717-25]. Listing 7 shows the last row of this worksheet.

**Listing 7: The last row in the worksheet csv-71717-25**

```
<row r="1048576" spans="3:7" ht="12.75"
    customHeight="1">
  <c r="C1048576" s="538"/>
  <c r="D1048576" s="537"/>
  <c r="E1048576" s="537"/>
  <c r="F1048576" s="537"/>
  <c r="G1048576" s="541"/>
</row>
</sheetData>
```

Unlike the whitespaces, these cells now contain no value and therefore do not need to be cross-referenced with the sharedStrings file. However, it is possible that such empty cells may have a background color assigned, as is the case with some cover sheets used for page design or for highlighting table headers. Additionally, these cells might have borders applied.

To determine this, one must examine the `s` attribute, which stands for the style index applied to the cell. The values `s="538"`, `s="537"`, and `s="541"` correspond to indices in a lookup table where all cell styles are stored. The styles matching these style indices can be found in the `xl/styles.xml` file. The relevant excerpts are shown in Listing 8.

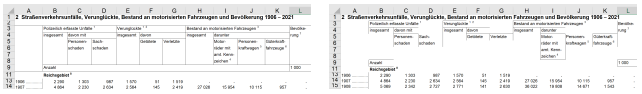
**Listing 8: Cell styles in styles.xml for csv-71717-25**

```
<cellXfs count="778">
  <!--538-->
  <xf numFmtId="0" fontId="0" fillId="0"
      borderId="0" xfId="0"/>
  <!--539-->
  <xf numFmtId="0" fontId="4" fillId="0"
      borderId="0" xfId="3"
      applyNumberFormat="1" applyFont="1"/>
  <!--542-->
  <xf numFmtId="0" fontId="1" fillId="0"
      borderId="0" xfId="1"
      applyNumberFormat="1" applyFont="1"
      applyAlignment="1">
    <alignment horizontal="right"/>
  </xf>
```

The XML structure within `<cellXfs>` contains formatting properties defined for cells in an *Excel* file. Each `<xf>` element represents a formatting rule. Similar to the Shared Strings file, formatting rules are indexed based on their order in the list, starting with the first at 0, the second at 1, and so on. For the IDs `s="538"`, `s="537"`, and `s="541"`, one must look for the 539th, 538th, and 542nd elements in the list, respectively. The attributes control the cell's appearance. The `numFmtId` attribute specifies the number format used for the cell, while `fontId` describes the font ID. `fillId` refers to the fill color, and `borderId` defines the border properties. The `xfId` references a format template from which the cell inherits properties. The attributes `applyNumberFormat`, `applyFont`, and `applyAlignment` indicate whether the corresponding number format, font, and text alignment are applied to the cell. If the `alignment` element is present, it specifies the text alignment within the cell, such as right alignment. This structure defines the formatting rules used for the display and layout of the cell contents.

Inspection of these styles reveals that these cells only apply styles that are not visible for empty cells. They use different fonts, and cell `G1048576` is additionally right-aligned. Therefore, all these cells are redundant and can be deleted accordingly.

Another case arises when the formatting rules include borders. If such cells are simply deleted, as occurred in previous versions of our script, issues arise, as seen in Figure 7 [31, Sheet: `2_(1)`]. In Figure 7a, all borders around the cells in the table header are visible. In Figure 7b, only the borders adjacent to cells with content remain visible. For example, the border between cells `A2` and `A3` is missing.



**(a) Original version with correct cell borders and spacing**

**(b) Corrupted version with removed borders and spacing**

**Figure 7: Erroneous removal of cells with borders and deletion of rows with custom heights**

Cell `A2` appears in the XML as shown in Listing 9.

**Listing 9: Cell A2 in sheet7.xml**

```
<c r="A2" s="577"/>
```

Here, the style ID is `s="577"`. It defines the underlying cell formatting, which in turn includes the ID for the border definition.

In the `styles.xml` file under the `<cellXfs>` node, we search for the `<xf>` node with ID 577, which corresponds to the 578th `<xf>` node. This is shown in Listing 10.

**Listing 10: Style with implicit ID 577 in styles.xml**

```
<xf numFmtId="0" fontId="8" fillId="2"
    borderId="5" xfId="0" applyFont="1"
    applyFill="1" applyBorder="1"
    applyAlignment="1"/>
```

This `<xf>` node has the `borderId` attribute set to index 5. Similar to cell styles, this index indicates that the 6th `<border>` node within the same `styles.xml` file is referenced.

Listing 11 shows the 6th `<border>` node with implicit ID 5.

**Listing 11: Border node with implicit ID 5 in styles.xml**

```
<borders count="12">
  <!-- ... -->
  <border>
    <left/>
    <right/>
    <top/>
    <bottom style="thin">
```

```
      <color indexed="64"/>
    </bottom>
    <diagonal/>
  </border>
```

Each <border> node contains the child nodes left, right, top, bottom, and diagonal. If these nodes lack attributes or child nodes, it means that the cell has no border. In this case, the bottom node has the attribute style set to "thin" and includes the color node with the attribute indexed="64", indicating that the cell has a thin black line at the bottom. Cells identified as having borders must not be deleted. Only cells that have neither content nor borders (i.e., their border nodes lack attributes indicating border presence) may be safely deleted.

## A.4 Inverted Fill

During the development of *Excel Shrink*, we made an interesting discovery. We found cells that had no content and no border but a fill definition that specified no fill should be applied. As a result, these cell definitions were deleted, and the cells turned red. We discovered that the entire row containing the cell had a red fill applied, and each cell within that row had a definition specifying that no fill should be applied to override the red fill color. As a user, one does not have any means to identify this by looking at the sheet. All the visible cells had the overridden fill color, and the rest of the cells were hidden behind hidden columns. Only after unhiding those columns did the issue become evident. Situations like these make preserving the visual appearance especially difficult.

We cannot simply decide to delete all cells that have a style applied that tells the cell to apply no fill because such a scenario might be useful in some situations. For example, if the whole row or column is intended to have a certain style and only some cells should apply no style as an exception to the rule. At the same time, in some scenarios, this might not be intended. The file becomes exceptionally large because some rows inadvertently have a style applied, and all cells in those rows up to the limit of possible columns define that this style should be overridden by no style.

To reduce the file size while preventing such changes to the display, we proceed as follows. First, we determine the style of the column and the row in which the cell is located. If either the column or row has a background color defined, we check whether the cell defines the same background color. In this case, we delete the cell. If the background color information of the cell differs from that of the column or row, we retain the cell. The same applies analogously to borders. If the column or row defines a border, we check whether the cell defines the same border. In this case, we delete the cell. If the border information of the cell differs from that of the column or row, we retain the cell.

# B DELETION OF EMPTY ROWS

Similar to cells without content but with visible formatting, rows may contain no defined cells but still have a defined height. Such rows must not be deleted because their height can affect the worksheet's layout. If such a row is deleted, an issue occurs as depicted in Figure 7b compared to the original in Figure 7a. The spacing between the header and the table shrinks because the intermediate row, which served as spacing between the header and the table, is deleted, and the default row height for the worksheet is applied, which is smaller than the custom height of the row. This row is shown in Listing 12.

**Listing 12: Row with custom height**

```
<row r="2" spans="1:12" ht="8.25"
    customHeight="1">
```

The attribute customHeight means that this row has been assigned a user-defined height, specified by the ht attribute: 8.25. Therefore, this row must not be deleted. However, during our examination of the worksheets, we found many that contained an immense number of empty rows, which had defined heights.

In the most extreme case in our analysis, a worksheet contained 1,048,576 defined rows. Of these, only 94 rows had content, which corresponds to approximately 0.00896% of the total rows [32, Sheet: Tabelle 1.2.1]. The remaining 1,048,482 rows had a defined height but were outside the data range.

**Listing 13: The last row in the worksheet**

```
<row r="1048576" ht="12.75" hidden="1"
    customHeight="1"/>
```

After the 96th row, which is the last one with content, there are 1,048,482 rows that are identical to the last row shown in Listing 13, except for the r attribute. These rows all have the same height and are hidden, as indicated by the hidden="1" attribute. Although they are not displayed to the user, they have a user-defined height of 12.75 units, which is why they are still stored in the file. Row 1,048,576 corresponds to the last row, which is the maximum number of rows allowed in *Excel*. If *Excel* allowed a higher number of rows, the wasted file storage would be significantly greater.

We cannot rely on a row simply not having a defined height to decide whether it can be deleted. However, we also cannot simply delete rows that have a user-defined height, as they may be important for the worksheet's layout. Therefore, we must find another method to identify empty rows that can be safely deleted.

To ensure that deleting empty rows does not alter the worksheet's visual representation, we adopt a strategy similar to that used for removing columns. First, we determine the actual range where values, background colors, or borders are defined. Then, we iterate through all empty rows and delete them only if they lie outside this range. This approach allows us to significantly reduce the file size of some worksheets without altering their content or appearance.

## B.1 Deletion of Redundant Print Areas and Print Titles

Excel allows users to define the area of a worksheet that should be printed on paper. This area can be manually defined or automatically set by *Excel*. In both cases, this area is stored in the workbook.xml file under the <definedNames> node [13]. Typically, this area is labeled as _xlnm.Print_Area and _xlnm.Print_Titles. However, these areas and titles are often also stored under the names Print_Area and Print_Titles. When *Excel* opens the file, it renames the entries Print_Area and Print_Titles to _xlnm.Print_Area and _xlnm.Print_Titles, respectively. This process creates a problem when there is already a *Defined Name* for the same worksheet

with both Print_Area and _xlnm.Print_Area, as seen in Listing 14[32, xl/workbook.xml].

**Listing 14: Redundant Print Area entries**

```
<definedName name="_xlnm.Print_Area"
    localSheetId="12">'Tabelle
    2.1.1'!$A$1:$N$64748</definedName>
<!-- ... -->
<definedName name="Print_Area"
    localSheetId="12">'Tabelle
    2.1.1'!$A$1:$N$93</definedName>
```

The same applies to Print_Titles and _xlnm.Print_Titles. In this case, the entries cannot be renamed because it would lead to redundant entries. Instead, *Excel* presents a dialog to the user upon opening the file, asking the user to select a new name for the *Defined Name*.

If the first entry is renamed and the user clicks OK, the dialog reappears for the next entry. This process repeats until all redundant entries for both Print_Area and Print_Titles are renamed. However, the user cannot simply enter the same name repeatedly to shorten this process, as *Excel* will prevent the same name from being used twice for different *Defined Names* on the same worksheet. Dieser Dialog verhinder auch, die Dateien mit VBA automatisiert zu öffnen um mittels des Plugins Inquire oder dem Excel Performance Tab die Dateigröße zu verringer, da beim öffnen jeder Datei über VBA dieser Dialog auftaucht und die automatisierung anhält.

In the files we examined, there were up to 60 redundant Print_Area and 55 Print_Titles entries. Renaming these entries offers no advantage to the user, as these entries are only stored in the *Defined Names* accessible via *Excel*'s Formulas → Name Manager. Users might think that they are repairing the *Print Areas* or Print Titles by entering those new names, but this is not the case because those *Print Areas* and Print Titles are already migrated correctly. In reality, the user only renames redundant *Defined Names*. To simplify the analysis and spare users the need to manually rename redundant entries, *Excel Shrink* was extended to automatically remove redundant entries.

## B.2 Reporting Reduction Results

To provide a breakdown of how many bytes were saved by deleting whitespaces, empty cells, columns, and rows, we compare the file size before and after each reduction step. For this purpose, we calculate the size of the worksheet by converting the XML structure into a string, as it would be stored in the file, and compute the length of this string. We then perform a reduction step and again calculate the length of the new string to determine the difference caused by the reduction. Finally, we calculate the percentage reduction relative to the original size. These results are saved in a CSV table. [3]

# C EVALUATION

In this evaluation, we processed all *Excel* files obtained from the German Federal Statistical Office (*Destatis*) [1, 2]. The complete details of how these files were acquired through a custom web scraping approach can be found in Appendix H.

Our primary goal was to quantify the extent of data reduction achieved by *Excel Shrink*, focusing on metrics such as overall size reduction, removal of empty cells, elimination of whitespace cells, deletion of rows outside the value range, and the removal of unnecessary columns. Figure 8 presents the distribution of reduction rates (in percentage) for individual worksheets across all these categories. The chart shows that some worksheets experienced reductions approaching 100% in file size. In particular, one worksheet exceeded a 99.99% reduction, 16 worksheets surpassed 99.9%, and 210 worksheets achieved reductions above 99%. Across all worksheets, the average reduction rate was 25%. Most of these gains were realized by removing rows that lay outside the range of actual data values.

To further assess the performance improvements enabled by *Excel Shrink*, we identified the largest available *Destatis* file.[2] Prior to processing, this file measured 85.2 MB in its compressed *Excel* (.xlsx) format and expanded to 911 MB when uncompressed. After applying *Excel Shrink*—a process that took approximately 935.5 seconds—the resulting compressed file size was reduced to 2.76 MB, with an uncompressed size of just 20.6 MB.

We then tested file opening times using the Dart excel package. Before shrinking, the first run took 794.7 seconds to open and read values from the file. On the second run, the process failed due to an out-of-memory exception. In contrast, after applying *Excel Shrink*, we successfully ran the test 10 consecutive times without error. The median time to open the shrunk file and read values was 17.90 seconds, demonstrating a substantial performance improvement.

## C.1 Summary of Findings

The evaluation demonstrates that the data reduction techniques employed by *Excel Shrink* are highly effective in eliminating redundant, empty, or irrelevant data across a variety of *Excel* worksheets. Significant reductions—exceeding 90% in several cases—were achieved in sheets containing a high proportion of empty cells, or irrelevant rows and columns. These results highlight the robustness of our reduction methods in processing diverse data formats and structures.

Moreover, the analysis underscores the importance of addressing different reduction factors independently, as their impact on file size varied. While empty cell removal provided substantial reductions in certain worksheets, others benefited more from the removal of rows or columns. Notably, whitespace removal had a smaller overall impact.

# D EXPERIMENTAL SETUP

To evaluate the behavior of *Excel* regarding how the files are interpreted and saved, as well as to test the behavior of the *Add-in* and the new *Check Performance* feature, we tested with two distinct versions of *Excel* from *Microsoft Office*: *Microsoft Office LTSC Professional Plus 2021* and *Microsoft Excel for Microsoft 365 MSO (Version 2409 Build 16.0.18025.20030) 64-bit*. These versions were selected to evaluate compatibility and performance across different Excel environments.Our test machine was an *Intel Core i7-4790K* quad-core CPU desktop machine running at 4.00 GHz with 16 GB of RAM.

[2]The file reference can be found here: [LINK TO THE FILE].

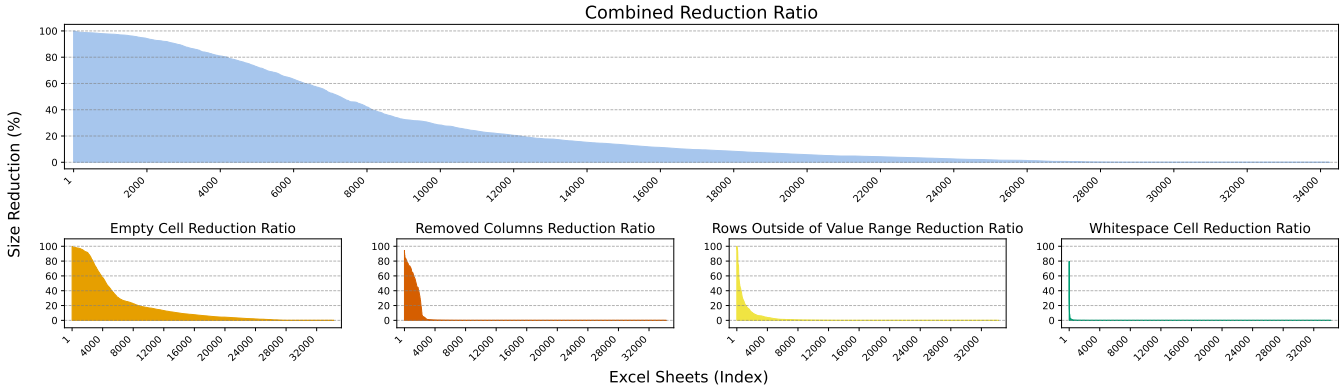



**Figure 8: Distribution of Size Reduction Percentages for Excel Sheets**

# E CONCLUSION

We have presented *Excel Shrink*, an open-source tool that significantly reduces the file sizes of *Excel* sheets from *Destatis* by eliminating unnecessary content. Our analysis identified common practices that lead to bloated *Excel* files and demonstrated how targeted removal of redundant data can achieve reductions of up to 99.99% without altering the spreadsheets' appearance.

Our tool outperforms existing solutions in both file size reduction and preservation of formatting. By making *Excel Shrink* available to the community, we hope to facilitate more efficient data processing and analysis of large-scale *Excel* datasets.

# F OLD BACKGROUND

## F.1 Limitations of Existing Solutions

Several proprietary tools and features, such as NXPowerLite Desktop, the Inquire Add-in, and the Excel Performance tab, attempt to mitigate file bloat. However, they suffer from notable shortcomings:

- **Closed Source and Lack of Transparency:** Existing tools do not disclose their underlying methods, making it difficult to understand the root causes of bloat or to improve upon their techniques.
- **Incomplete Coverage:** They often fail to address all sources of unnecessary content. For example, redundant column definitions may remain uncorrected, leaving substantial portions of file bloat unresolved.
- **Visual Alterations:** Some solutions can significantly alter the spreadsheet's original appearance, which is unacceptable for users who rely on specific formatting or corporate branding.
- **Limited Automation:** Due to the lack of batch processing or command-line interfaces, integrating these tools into automated workflows or pipelines (e.g., for cloud uploads or large-scale data processing) is challenging or impossible.

## F.2 Our Approach and Contributions

Motivated by these gaps, we conducted a systematic investigation into the root causes of Excel file bloat, focusing on previously undocumented inefficiencies and the limitations of existing solutions. We then developed *Excel Shrink*, an open-source tool designed to effectively reduce file sizes while preserving the spreadsheet's visual appearance. Unlike proprietary offerings, *Excel Shrink* provides a transparent, platform-independent solution that does not require Microsoft Excel itself. It can also be integrated into automated pipelines, enabling seamless batch processing and large-scale deployment. For those preferring a more user-friendly interface, we offer *Excel Shrink UI* [5], which eliminates the need for Python installation or command-line interactions.

Our contributions are as follows:

- **Comprehensive Analysis:** We reveal the root causes of file bloat in Destatis Excel files, identifying previously undocumented issues such as unnecessary column definitions.
- **Critical Evaluation of Existing Tools:** We show why current proprietary solutions fail to fully resolve file bloat and how they may inadvertently alter the spreadsheet's appearance.
- **Novel Open-Source Solution:** We introduce *Excel Shrink*, which effectively addresses all identified causes of bloat, preserves visual formatting, supports automation, and is not tied to a specific operating system or to Microsoft Excel.
- **Empirical Validation:** We evaluate *Excel Shrink* on a comprehensive set of Destatis Excel files, demonstrating that it can reduce file sizes by up to 99.99% without compromising usability or appearance.

By thoroughly examining hidden redundancies and providing a transparent, open-source solution, our work fills a significant gap in spreadsheet optimization research and practice.

# G OLD RELATED WORK

Several tools and techniques have been developed to reduce *Excel* file bloat, including proprietary solutions such as *NXPowerLite Desktop*, the *Excel Inquire* Add-in, and the built-in *Check Performance* feature in recent versions of *Excel*. While these tools can alleviate certain issues, they also exhibit notable limitations.

## G.1 NXPowerLite Desktop

*NXPowerLite Desktop* is a commercial application primarily focused on compressing embedded images in *Excel* files [27]. Although it

can remove some empty cells, it disregards other sources of file bloat such as cells containing only whitespace and redundant column definitions.

The tool offers limited batch functionality, allowing users to select and process multiple files simultaneously, but only up to a maximum of 20 files at once. Being closed-source, it does not provide a means to adjust this artificial limit, further motivating the need for a more flexible, open-source solution.

## G.2 Excel Inquire Add-in

The *Excel Inquire* Add-in, available only in *Office Professional Plus* and *Microsoft 365 Apps for Enterprise*, can remove some redundant content [29]. However, it lacks comprehensive batch processing capabilities because it requires manually opening each *Excel* file before applying the add-in. This approach is impractical for large datasets.

*Excel Inquire* ignores whitespace cells but can delete empty cell, column, and row definitions. Yet, in certain cases, a significant number of unnecessary elements remain. For example, consider a worksheet serving as a cover sheet, where only the first 77 rows are used and the remaining rows (78 to 65533) are hidden. Despite being hidden, all these rows and cells are defined in the XML structure, as shown in Listing 15.

**Listing 15: Row 65533 in sheetX.xml**

```
<row r="65533" spans="1:7" ht="0"
    hidden="1" customHeight="1">
  <c r="A65533" s="68" />
  <c r="B65533" s="68" />
  <c r="C65533" s="68" />
  <c r="D65533" s="68" />
  <c r="E65533" s="68" />
  <c r="F65533" s="68" />
  <c r="G65533" s="68" />
</row>
</sheetData>
```

When applying the *Excel Inquire* Add-in to such worksheets, these hidden rows and cells remain. A dialog box reports that because the default row height is `0`, the tool cannot clean the sheet, causing the operation to be skipped entirely.

## G.3 Excel Check Performance

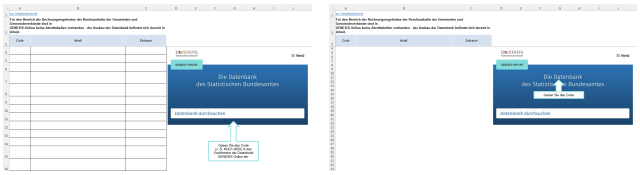
The *Check Performance* feature, introduced in *Excel* for the Web and recent desktop versions, can achieve substantial file size reductions [41, 42]. However, it often removes cells and rows that influence the spreadsheet's layout and formatting, thereby altering its visual appearance. This is unacceptable in contexts where maintaining the original look and structure of the spreadsheet is essential.

Figure 9 illustrates how *Check Performance* modifies a cover sheet by removing empty rows. While such modifications might be acceptable in some instances, they become problematic when these empty rows serve a functional purpose, such as aligning the layout for printing on A4 pages. In the example shown, the cells responsible for providing whitespace on the cover sheet (Figure 9a) are removed, as shown in Figure 9b [31].


**(a) Before Applying Check Performance** **(b) After Applying Check Performance**

**Figure 9: Comparison of a Cover Sheet Before and After Applying Check Performance**

As another example, consider a form-like structure composed of several empty cells arranged to form a table, anticipating future data entry [34]. *Check Performance* removes all borders outside the active data region, resulting in the scenario shown in Figure 10, where the intended table layout is lost.



**(a) Before Applying Check Performance** **(b) After Applying Check Performance**

**Figure 10: Removal of Borders Defining an Empty Table Structure**

# H SCRAPING DESTATIS

The *Destatis* website does not provide an API for searching publications, making it necessary to interact with the website directly. To automate this process, we developed a web scraper that searches for relevant publications while minimizing the number of requests to the server. The web scraping and downloading process is outlined in Algorithm 5. The URL structure of the *Destatis* website is as follows, with query parameters appended to the base path:

- /SiteGlobals/Forms/Suche/Expertensuche_Formular.html:
  This is the base path of the URL, which points to the expert search form on the *Destatis* website. It serves as the primary endpoint where the search query is submitted.
- templateQueryString=2000:
  This query parameter specifies the search year. In this case, the search is focused on publications from the year 2000. By adjusting this value, different years can be searched.

- gtp=474_list%253D2:
  This parameter handles pagination. It points to a specific page of search results. The value `%253D` is a doubly encoded sequence, where `%253D` is the URL-encoded form of `%3D`, which itself encodes the equal sign (=). After decoding twice, the query parameter becomes `gtp=474_list=2`. The number after the second equal sign corresponds to the current page index in the pagination of the search results.
- documentType_=publication:
  This parameter filters the results by the type of document. By setting this to `publication`, the search is limited to publications, where *Excel* files are usually attached.
- resultsPerPage=100:
  This parameter determines how many results should be displayed per page. In this case, it is set to `100`, which is the maximum number of results that can be retrieved in a single page request.

Through extensive testing, we discovered that *Excel* files are exclusively found in the "Publication" document type. By using the GET parameter `documentType_=publication`, we were able to significantly narrow down our search results.

Initially, we assumed that searching for the term `xlsx` would identify all publications containing *Excel* file attachments. However, this search only returned about 100 results, far fewer than the actual number of *Excel* files available on the *Destatis* website. Furthermore, there is no option to retrieve a complete list of all publications ever created. To overcome this, we refined our strategy by searching based on publication years. A publication should appear in the search results when queried by its publication year. Although this approach returns some irrelevant results—where the publication year might appear in the text or a file attachment—the number of requests is greatly reduced by caching and reusing links to the *Excel* file attachments. This ensures that the same files are not downloaded multiple times.

The search results on the *Destatis* website, like many others, are paginated, with the interface offering the ability to display 10, 20, or 30 entries per page. The number of results per page can be adjusted using the GET parameter `resultsPerPage=30`. While the interface does not provide an option for more than 30 results per page, we found that manually setting this parameter allowed us to request up to 100 results per page. Through testing, we confirmed that 100 is the maximum number of results that can be returned at once. We set the `resultsPerPage` parameter accordingly to reduce the number of requests further.

Once the results for a page are fetched by the web scraper, we simulate clicking the "next page" link to load subsequent results. The XPath expression `//a[@class='forward button']/@href` is used to identify the link using the CSS class `forward button`.

Once we have queried each publication year from `2000` to `2024`, and the file attachments have been extracted from the result pages until no further "next page" button is available, the scraping process for the *Excel* files is complete.

## H.1 Respecting Crawl Delay

To minimize the load on the *Destatis* website, we adhered to the `crawl-delay` directive specified in the site's `robots.txt` file [35], pausing for 30 seconds between requests. This delay ensures that our scraper operates within the guidelines provided by *Destatis*.

**Algorithm 4** Processing and Cleaning Excel Files in Place

**Input:**
`InP` ← Excel file path
`OutP` ← Output path

`InArc` ← Open the ZIP archive `InP` for reading
`OutArc` ← Create a new ZIP archive for writing
`ShStr` ← Extract Shared strings from `sharedStrings.xml`
`Sty` ← Extract Styles from `styles.xml`
`SF` ← List of all sheet XML files in `Arc`
`NSF` ← List of non-sheet files in `Arc`

`wvp` ← dynamicly create pattern based on `ShStr` to identify cell values that are whitespace
`icp` ← dynamicly create pattern based on `Sty` to identify cells that are invisible
`sdcp` ← dynamicly create pattern based on `Sty` to identify cells their styles depend on the style of their column and or row style

**for** each sheet file `sf` in `SF` in parallel **do**
  Clean sheet file `sf` streambased using `wvp`, `icp`, `sdcp` and `Sty`
  Write cleaned sheet back to ZIP archive `OutArc`
**end for**

**for** each non-sheet file `nsf` in `NSF` **do**
  **if** `nsf` is `workbook.xml` **then**
    Clean workbook metadata with DOM
    Write cleaned workbook to ZIP archive `OutArc`
  **else**
    Copy file non-sheet file `nsf` unchanged to ZIP archive `OutArc`
  **end if**
**end for**
**Output:** Cleaned Excel file

# I SURPRISE

*Check Performance* was capable of outperforming *Excel Shrink* in specific cases without altering the worksheet's appearance. Nevertheless, it modified the worksheet in a manner that required users to perform multiple steps to revert the changes made by *Check Performance*. This issue arose when a worksheet contained an actual value in a hidden row, and between this row and the next visible row, there were thousands of additional hidden rows without any values. For example, we had a worksheet whose visible content extended only to row `94`. After making the invisible rows visible to verify our observations from the XML file of the sheet, we found that in row `65536`, column `A`, the number `3` was stored, presumably by accident [32, Sheet: `Tabelle 2.1.1`].

In such scenarios, *Excel Shrink* is unable to remove the hidden rows because our strategy involves removing only content outside the range of actual values and visible formatting. Since the last row within the user-designated area contains an actual value, all intervening rows must be preserved. In contrast, *Check Performance*

**Algorithm 5** Web Scraping and Downloading Excel Files with Caching

**Input:** Base URL `B`, form URL `F`, download folder `D`, CSV cache file `C`, crawl delay (30 seconds)
**Initialize:** Cached URLs set `U`, and load file records from CSV `C`
Create folder `D` if it does not exist
{— Scraping Process —}
**for** each year in years **do**
  Construct the search URL using year and base parameters
  **while** there are more pages to fetch **do**
    Send HTTP GET request to search URL
    Parse HTML to extract `.xlsx` links
    Filter links that are not already in the cache `U`
    **for** each new link **do**
      Extract file name from link
      Add new link and file name to file records
      Append new records to cache `C`
    **end for**
    Save the updated file records back to CSV `C`
    Check for the next page button
    **if** next page exists **then**
      Update URL to the next page
      Wait for `CRAWL_DELAY` seconds
    **else**
      Exit the loop
    **end if**
  **end while**
**end for**
{— Download Process —}
**for** each new file record **do**
  Download file from the URL
  Save it in the download folder `D`
  Wait for `CRAWL_DELAY` seconds
**end for**

removes all these hidden rows. This action alone would typically result in the previously hidden and now deleted rows becoming visible again. To prevent this, *Check Performance* employs a workaround by setting the row height to zero for the entire worksheet. This creates the illusion that the rows are deleted when, in fact, they are merely not visible due to having no height.

However, this approach presents a significant drawback for users. Once users select all rows and attempt to make them visible again, the previously hidden rows remain invisible. To actually make all rows visible, users must select all rows in the worksheet and assign a new height. This method is disadvantageous because it also alters the height of rows that users may have previously customized. To preserve these custom height settings, users must follow a specific procedure: first, select all rows using the *Select All* button, then hold down the *Control* key and deselect all visible rows except the last one. Consequently, only the last visible row and all hidden rows remain selected. Users can then adjust the height of all previously invisible rows using the resize handle of the last visible row, thereby making them visible again.

Given that users may be surprised when attempting to make the rows visible in the usual manner but find that the rows remain hidden, we decided not to adopt the same strategy as *Check Performance*. Considering that the steps required to actually make these rows visible involve a learning curve and can be time-consuming—especially when the worksheet already contains several visible cells requiring multiple scrolls to deselect the hidden rows—we opted to retain these rows to avoid imposing additional complexity and inconvenience on the user.

# J DETERMINING THE SHEET NAME

To provide the user with a breakdown of the reduction per worksheet, it is necessary to determine the worksheet titles. However, through the worksheet files, we only know an ID indicated in the filename, such as `sheet1.xml`, `sheet2.xml`, and so forth. Unfortunately, we cannot determine the sheet title directly from the sheet file because the title is only associated with the individual sheets in the `xl/workbook.xml` file. This association is shown in Listing 16 [30]. The `sheetId` attribute should be ignored; the ID that maps the sheet title to the corresponding `sheetX.xml` is the `r:id` attribute. Since each value in this attribute starts with `rId`, the ID must be extracted by removing the first three characters. These IDs then provide a unique mapping to the worksheet titles, which can be used for output.

**Listing 16: workbook.xml mapping IDs to worksheet titles**

```
<sheets>
  <sheet name="Titelseite" sheetId="10"
      r:id="rId1"/>
  <sheet name="Inhalt" sheetId="11"
      r:id="rId2"/>
  <sheet name="Erläuterungen" sheetId="3"
      r:id="rId3"/>
  <sheet name="Allgemeines" sheetId="4"
      r:id="rId4"/>
  <sheet name="0901_T_D" sheetId="12"
      r:id="rId5"/>
  <sheet name="0901_T_BW" sheetId="13"
      r:id="rId6"/>
```

# K REDUCTION RATIO

The evaluation results focus on the top five files for each reduction factor, as shown in Table 1.

## K.1 Overall Reduction Ratio

The top five worksheets with the highest overall reduction ratios are all from the same file, with reduction ratios ranging between 99.95% and 99.99%. This indicates that almost all data in these sheets consisted of file bloat that was successfully eliminated. The repetition of this file in the subsequent table suggests that the high reduction is primarily attributable to a specific reduction category.

## K.2 Rows Outside of Value Range Reduction Ratio

The majority of unnecessary data consisted of rows outside the value range, with reduction ratios between 99.94% and 99.99% in the

**Table 1: Top Sheets by Each Reduction Factor**

| File | Sheet | Reduction Factor Value (%) |
|---|---|---|
| **Top 5 rows for Reduction Ratio** | | |
| verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx | Technisch_bedingte_Leerseit | 99.99 |
| verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx | 4.6_(3) | 99.96 |
| verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx | 3.1_(7) | 99.96 |
| verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx | 5.1.4_(2) | 99.95 |
| verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx | 3.1_(1) | 99.95 |
| **Top 5 rows for Rows Outside of Value Range Reduction Ratio** | | |
| verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx | Technisch_bedingte_Leerseit | 99.99 |
| verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx | 3.1_(7) | 99.96 |
| verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx | 4.6_(3) | 99.95 |
| verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx | 3.1_(1) | 99.95 |
| verkehrsunfaelle-zeitreihen-xlsx-5462403.xlsx | 10.1 | 99.94 |
| **Top 5 rows for Empty Cell Reduction Ratio** | | |
| statistischer-bericht-mikrozensus-h...-2010300217005.xlsx | Erläuterung zu CSV-Tabellen | 99.91 |
| statistischer-bericht-mikrozensus-h...erstergebnisse.xlsx | Erläuterung zu CSV-Tabellen | 99.91 |
| statistischer-bericht-mikrozensus-h...-endergebnisse.xlsx | Erläuterung zu CSV-Tabellen | 99.91 |
| statistischer-bericht-migrationshin...-2010220227005.xlsx | Erläuterung zu CSV-Tabellen | 99.90 |
| statistischer-bericht-migrationshin...-2010220227005.xlsx | Erläuterung zu CSV-Tabellen | 99.90 |
| **Top 5 rows for Removed Columns Reduction Ratio** | | |
| strauchbeerenanbau-2030319227005.xlsx | DE3_2_(4) | 94.38 |
| strauchbeerenanbau-2030319227005.xlsx | DE3_2_(3) | 93.41 |
| strauchbeerenanbau-2030319227005.xlsx | DE3_2 | 93.37 |
| strauchbeerenanbau-2030319227005.xlsx | DE3_2_(2) | 92.43 |
| fallpauschalen-krankenhaus-2120640167005.xlsx | Erläuterungen Fehlerkorrektur | 91.11 |
| **Top 5 rows for Whitespace Cell Reduction Ratio** | | |
| statistischer-bericht-berufliche-sc...-5211004227005.xlsx | 21121-17 | 79.71 |
| statistischer-bericht-verkehr-2080110231075.xlsx | csv-46251-28 | 44.68 |
| ueberschuldung-2150500217005.xlsx | 5.1 | 29.97 |
| rohholz-holzhalbwaren-9030001207005.xlsx | Seite_5 | 27.91 |
| rohholz-holzhalbwaren-9030001207005.xlsx | Seite_4 | 27.61 |

top five files. This suggests that the largest portions of redundant data were predominantly from a single category rather than a combination of different categories.

## K.3 Empty Cell Reduction Ratio

Empty cell removal achieved significant reductions, with ratios between 99.90% and 99.91% in the top five files. Notably, three different files were affected, although the worksheets shared the same name. This suggests that these explanatory worksheets might have been duplicated from one *Excel* file to another, or the entire file was duplicated, leading to a similar number of empty cells across them.

## K.4 Removed Columns Reduction Ratio

Removing unnecessary column definitions resulted in reduction ratios ranging from 92.43% to 94.38% in the top five worksheets. Similar worksheet names were again noticeable, implying that duplicated worksheets carried over a significant amount of file bloat due to unused column definitions.

## K.5 Whitespace Cell Reduction Ratio

Whitespace cell removal was the least effective in reducing file size. The top sheet still held a notable 79.71% of the file size attributed to whitespaces, while the second entry dropped to 44.68%, and the last three entries were around one-third. With the exception of the last two files, the top five were distinct. However, the similar worksheet names and high proportion of whitespace suggest that redundant content might have been carried over to other worksheets through duplication.